



OFFICIAL REFERENCE MANUAL

ModelScript Engineering Toolchain

ModelScript

The Polyglot Cyber-Physical Modeling, Simulation & Formal Verification Environment

Architecture, Languages & Algorithmic Foundations

Edition & Target

2026 Comprehensive Edition
WebAssembly (WASM) & Node.js ≥
24

Open Source License

GNU AGPL v3.0
github.com/modelscript/modelscript

Documentation Portal

docs.modelscript.org
Compiler & Engine Core

Table of Contents

Part I: Getting Started & User Guide

1. Introduction
2. Installation & Requirements
3. Getting Started

Part II: Compiler & Pipeline Architecture

4. Compiler Pipeline Overview
5. WASM GLR Native Parser
6. Salsa Query Engine
7. Linear DAE Arena
8. Triple Graph Grammars
9. Nelson-Oppen Coordinator
10. Simulation Solvers

Part III: Polyglot Languages & Formats

11. Language Support Overview
12. Modelica Physical Modeling
13. SysML v2 & KerML
14. STEP CAD (ISO 10303)
15. OWL2 Ontologies
16. CSV Telemetry
17. CFD (SU2 & OpenFOAM)
18. FEA Structural Analysis
19. OpenSCAD CSG Geometry
20. SSP System Packaging

Part IV: Comprehensive Algorithms Catalog

21. Algorithms Overview & Map
22. Pantelides Index Reduction
23. Tarjan BLT Partitioning
24. Cellier-Elmqvist Tearing
25. Acausal Connector Balancing
26. Automatic Differentiation
27. Numerical ODE/DAE Solvers
28. Optimization Algorithms
29. Nelson-Oppen & SMT Solvers
30. CHC & IC3 Verification
31. Abstract Interpretation Domains
32. Graph Rewriting & E-Graphs
33. Linear Memory Structures
34. Scientific Computing & Surrogates
35. SDP, BVP & Geometric CSG
36. Diagram Layout & Routing

Part V: Command-Line Interface (msc)

37. CLI Overview & Flags
38. Command Reference

Part VI: Reference & Diagnostics

39. Compiler Linter Rules
40. Standards & Export Formats

PART I

Getting Started & User Guide

CHAPTER 1

Introduction

Welcome to **ModelScript**, a powerful, modern, and polyglot modeling environment designed for the future of systems engineering.

What is ModelScript?

ModelScript is a unified workspace designed to bridge the gap between different systems modeling languages and 3D CAD representations. It currently provides deeply integrated support for:

- **Modelica**: An object-oriented, declarative, multi-domain modeling language for component-oriented modeling of complex systems.
- **SysML v2**: The next-generation systems modeling language for systems engineering.
- **STEP / CAD**: Seamless integration of standard 3D CAD files for visual simulation and binding.

Core Features

- **Blazing Fast LSP**: Powered by native WebAssembly GLR incremental parsers and a linear memory CST symbol indexer, providing sub-millisecond autocomplete, diagnostics, and symbol resolution.
- **Semantic Diff Engine**: Unlike standard text diffs, ModelScript understands the AST of your models and provides semantic diff annotations directly in your editor.
- **Unified Library Explorer**: A seamless sidebar for viewing both Modelica and SysML2 libraries with zero context switching.
- **Dynamic CAD Viewer**: Associate Modelica variables with 3D components for dynamic, in-editor animations and visual debugging.

Ready to get started? Head over to the [Installation guide](#).

CHAPTER 2

Installation

ModelScript can be installed as a VS Code extension for end users, or set up locally for developers looking to contribute.

For Users

The easiest way to use ModelScript is via the VS Code Marketplace.

1. Open VS Code.
2. Go to the Extensions tab (`Ctrl+Shift+X`).
3. Search for **ModelScript**.
4. Click **Install**.

The extension bundles the Language Server, the 3D Viewer, and all necessary polyglot tools out-of-the-box.

For Developers

To set up ModelScript for local development, you will need **Node.js ≥ 24** (matching `.nvmrc`) and `npm` . Docker is optional and only required if running backend database/API services.

1. Clone the Repository

```
git clone https://github.com/modelscript/modelscript.git
cd modelscript
```

2. Install Dependencies

We use `npm` workspaces to manage our monorepo packages.

```
npm install
```

3. Build the Monorepo

ModelScript compiles its native WebAssembly GLR parsers, AssemblyScript flattening kernels, and TypeScript packages via Nx:

```
npm run build:libs    # Build core compiler packages and language runtimes
npm run build         # Build all monorepo targets
```

4. Run the Dev Server

You can launch the IDE and the local web environment via:

```
npm run dev
```

This will spawn the compiler backend, the LSP workers, and open the web UI in your browser.

CHAPTER 3

Getting Started

This guide will walk you through creating your first ModelScript project, utilizing the polyglot capabilities for Modelica and CAD.

1. Creating a Workspace

1. Open an empty folder in VS Code.
2. Run the command `ModelScript: Initialize Workspace` from the command palette (`Ctrl+Shift+P`).
3. This creates a standard `.modelscript` configuration directory for your project.

2. Writing Your First Modelica Class

Create a new file called `BouncingBall.mo`:

```
model BouncingBall
  parameter Real e = 0.7 "Coefficient of restitution";
  parameter Real g = 9.81 "Gravity acceleration";
  Real h(start=1) "Height of ball";
  Real v(start=0) "Velocity of ball";
equation
  der(h) = v;
  der(v) = -g;
  when h <= 0 and v < 0 then
    reinit(v, -e * pre(v));
  end when;
end BouncingBall;
```

You should instantly see syntax highlighting and language server features.

3. Integrating 3D CAD

If you have a STEP file (e.g., `ball.step`), you can map the 3D model to the simulation variable `h` using a `DynamicSelect` annotation:

```
model BouncingBall
// ... parameters ...
Real h(start=1) "Height of ball" annotation(
  CAD(
    shape="ball.step",
    transform=DynamicSelect(
      translate(0, 0, 0),
      translate(0, h, 0)
    )
  )
);
// ... equations ...
end BouncingBall;
```

Open the `BouncingBall.mo` file, and click the **Open 3D Viewer** icon in the editor toolbar to see the ball respond to the Modelica equations!

PART II

Compiler & Pipeline Architecture

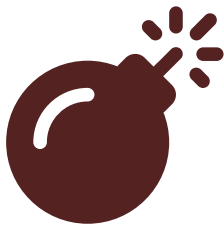
CHAPTER 4

Compiler Architecture Overview

ModelScript is an open-source, web-native polyglot compiler and multi-domain engineering intelligence platform. Designed to bridge the historical silos between system architecture, physical simulation, 3D CAD geometry, continuum analysis (FEA/CFD), and formal verification, ModelScript operates on a unified, high-performance digital thread.

End-to-End Compilation Pipeline

The compilation pipeline transforms polyglot source code into low-level, high-efficiency representations in linear WebAssembly memory without intermediate AST object allocations.



Syntax error in text
mermaid version 11.16.1

Key Architectural Principles

1. Data-Oriented Struct-of-Arrays (SoA) in Linear Memory

Traditional compilers instantiate millions of object-oriented AST nodes, leading to massive memory overhead, pointer chasing, and garbage collection pressure. ModelScript lowers expressions, equations, and statements directly from native WebAssembly parser buffers into `DAEBuilder` integer handles (`ExprId` , `EqId` , `VarId`) packed into contiguous typed arrays.

2. Zero-Allocation Incremental GLR Parsing

Grammars in `@modelsript/dsl` compile ahead-of-time to native WebAssembly GLR (Generalized LR) parsing engines. Tokens stream directly through linear memory, resolving ambiguous and non-deterministic grammar constructs without back-tracking allocations.

3. Demand-Driven Salsa Incremental Computation

All name resolution, component specialization, and inheritance hierarchies are governed by a Salsa-inspired query engine. Queries cache results and track fine-grained dependency edges:

fetch → deep_verify → execute → backdate

Modifying a single parameter or equation re-evaluates only transitively invalidated DAE components.

4. Coordinated Nelson-Open Multi-Theory Solvers

Formal verification combines distinct formal domains (Description Logic, non-linear interval arithmetic, continuous zonotope reachability, and 3D CAD collision checks) through an extensible Nelson-Open theory coordinator using CDCL(T) case splitting.

Monorepo Package Matrix

Package	Workspace	Role
@modelscript/dsl	packages/dsl	Grammar combinators, WASM GLR parser compiler (<code>buildParser</code>), TGG compiler, and E-graph rewriting
@modelscript/runtime	packages/runtime	DAEBuilder , linear memory CST arenas, Salsa QueryEngine , and Nelson-Open theory coordinator
@modelscript/simulate	packages/simulate	SUNDIALS CVODE/IDA numerical integrators, WebGPU batched ODE solver, and surrogate ROM models
@modelscript/lsp	packages/lsp	Multi-language Language Server Protocol server and background worker pool
@modelscript/diagram	packages/diagram	Polyglot diagram engine, auto-layout, and SVG rendering
@modelscript/exchange	packages/exchange	FMI 2.0/3.0 FMU export/import and SSP container packaging
@modelscript/cad	packages/cad	CAD engine: CSG primitives, OpenCascade kernels, and STEP B-Rep parsing
@modelscript/mcp	packages/mcp	Model Context Protocol server exposing compiler and simulation tools to AI agents

CHAPTER 5

WebAssembly GLR Incremental Parser

ModelScript avoids external runtime parser dependencies. Instead, every supported language in `languages/` is parsed using a custom, ahead-of-time (AOT) compiled **Generalized LR (GLR)** parser executing directly in WebAssembly (`parser.wasm`).

The parser compiler is implemented in `@modelscript/dsl` under `packages/dsl/src/codegen/`.

Why GLR in WebAssembly?

1. **Handling Complex Engineering Grammars:** Engineering modeling languages like Modelica and SysML v2 possess ambiguities that cannot be parsed by strict deterministic LR(1) or LALR parsers. The GLR algorithm handles non-deterministic grammar paths by branching parse stacks and pruning invalid branches dynamically.
2. **Zero-Allocation Memory Footprint:** Rather than allocating JavaScript AST object wrappers for every identifier and operator, tokens and CST nodes are laid out in a flat WebAssembly linear memory arena.
3. **Web-Native Performance:** The compiled `parser.wasm` executes at near-native speeds in both Node.js server environments and client-side browser contexts (such as the IDE and Morsel).

Parser Initialization

In TypeScript or AssemblyScript code, parsers are loaded and registered via `createWasmParser` :

```
import { createWasmParser } from "@modelscript/modelica/parser";
import { Context } from "../src/context.js";

// Load the compiled WebAssembly parser binary
const modelicaWasmPath = require.resolve("@modelscript/modelica/parser.wasm");
const { parser } = await createWasmParser(modelicaWasmPath);

// Register file extension mapping in the compilation Context
Context.registerParser(".mo", parser);
```

Linear CST Memory Layout

When a source file is parsed, the parser allocates a continuous buffer of node headers in linear memory. Each CST node is referenced by a 32-bit integer pointer (`u32` node offset) rather than an object reference:

Linear CST Node Arena			
Field	Offset	Type	Description
typeId	0	uint16	Node type code
flags	2	uint16	Semantic flags
startByte	4	uint32	Start byte pos
endByte	8	uint32	End byte pos
firstChild	12	uint32	Offset to child
nextSibling	16	uint32	Offset to sib

Direct Field Accessors (`cst`)

High-performance accessors read typed node properties directly from the linear buffer:

```
import { Cst } from "@modelsript/modelica/language";

// Accessing child fields without constructing intermediate wrappers
const comp = Cst.ComponentClause.componentDeclarationList(node);
for (const decl of comp) {
  const nameNode = Cst.ComponentDeclaration.declaration(decl);
  const identifier = db.ast.text(nameNode);
}
```

Incremental Re-Parsing

When editing code in the Language Server Protocol (LSP) or Web IDE, the GLR parser performs incremental tree re-use:

- Text edits pass line and character delta offsets.
- Unaffected subtrees in linear memory remain unchanged and are spliced into the new parse tree.
- Syntax diagnostics are generated instantly during parsing and stored in the diagnostics arena.

CHAPTER 6

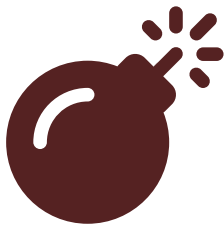
Salsa Incremental Query Engine

ModelScript's semantic analysis and symbol resolution pipeline is driven by an incremental query computation engine inspired by the open-source [Salsa](#) incremental query framework.

The engine lives in `@modelscript/runtime` and provides memoized, demand-driven evaluation across multi-file engineering workspaces.

The Query Life Cycle

Every query executed in ModelScript follows a 4-phase lifecycle to ensure maximum cache re-use and correct invalidation:



Syntax error in text
mermaid version 11.16.1

The 4 Stages

1. **fetch** : Checks if the query result exists in the memoization table.
 2. **deep_verify** : If the workspace revision has advanced, transitively checks whether any input queries or files that contributed to this result have actually changed.
 3. **execute** : Re-runs the query definition if inputs have genuinely changed, tracking any newly accessed queries.
 4. **backdate** : If re-executing the query produces an output value identical to the previous result, the query's revision is backdated to prevent cascading invalidation down the dependency tree.
-

Key Compiler Queries

The query engine handles semantic resolution, type checking, and specialization:

Query	Scope	Description
<code>instantiate</code>	Class	Resolves all direct components, inherited elements, and nested declarations of a class.
<code>classInstance</code>	Component	Specializes a generic class definition with specific modifier expressions.
<code>resolveSimpleName</code>	Scope	Performs hierarchical lexical scoping and imports resolution.
<code>variability</code>	Component	Computes effective variability: <code>constant</code> , <code>parameter</code> , <code>discrete</code> , or <code>continuous</code> .
<code>causality</code>	Component	Computes signal causality: <code>input</code> , <code>output</code> , or <code>internal</code> .
<code>effectiveModification</code>	Hierarchy	Resolves and merges class modification clauses across multi-level inheritance graphs.

Salsa & Linear CST Synergy

AST wrapper nodes are **never** stored inside the Salsa cache. Storing heavy AST objects would duplicate memory and defeat garbage collection guarantees.

Instead:

- The Salsa cache stores semantic integer handles and primitive metadata (e.g. `symbolId` , resolved `TypeInfo` , variability enums).
- Concrete CST nodes are retrieved on-demand from linear memory using `db.cstNode(symbolId)` only when lowering or evaluating equations.

CHAPTER 7

Linear Memory DAE Arena & Lowering

The Differential Algebraic Equations (DAE) representation in ModelScript is implemented via `DAEBuilder` in `@modelscript/runtime` (`packages/runtime/src/dae/wasm_dae.ts`).

Unlike traditional compilers that construct linked expression trees using heap-allocated objects, `DAEBuilder` operates strictly on **flat, contiguous typed arrays** (Struct-of-Arrays) in WebAssembly linear memory.

Arena Struct-of-Arrays (SoA) Layout

The arena maintains three primary contiguous arrays:

DAEBuilder Linear Memory		
Buffer	Element Size	Contents
<code>varData</code>	32 bytes	Name, type, variability, causality, start value, flags
<code>exprData</code>	16 bytes	OpKind, data1, leftExprId, rightExprId
<code>eqData</code>	24 bytes	EqKind, lhsExprId, rhsExprId, flags

Every variable, expression, and equation is assigned a persistent 32-bit integer handle:

- `varId` : Handle to a variable in `varData`
- `ExprId` : Handle to an expression node in `exprData`
- `EqId` : Handle to an equation in `eqData`

Lowering Pipeline

Flattening a high-level Modelica model into linear DAE form proceeds in two coordinated layers:

Layer 1: Components & Hierarchy

Reads Salsa queries (`instantiate` , `classInstance` , `variability`) to allocate variables and parameters in `DAEBuilder` :

- Scalar variables (`Real` , `Integer` , `Boolean` , `String`)

- Array dimensions and indices
- Start values, nominals, and min/max boundaries

Layer 2: Equations & Statements

Walks the linear CST of the class body (`db.cstNode(classId)`):

- Lowers binary, unary, and call expressions directly into `ExprId` nodes in linear memory.
- Emits equality equations (`lhs = rhs`), initial equations, and algorithm statements.

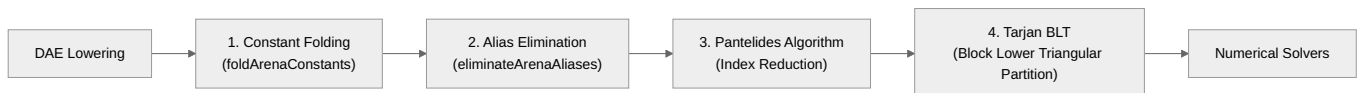
Expression Lowering Rules

To match OpenModelica's canonical output, the flattener applies specific normalization rules:

1. **Real Coercion in Binary Operations:** If one operand of an arithmetic operator is `Real` and the other is `Integer`, the integer operand is automatically cast to `Real` using `castToRealExpr()`.
2. **Negation Distribution:** Expressions of the form `-(a * b)` are rewritten canonically to `(-a) * b`.
3. **Binding Literal Kind Matching:** When a binding expression is lowered, `addIntLiteral()` is used for `VarType.Integer` and `addRealLiteral()` for `VarType.Real` (e.g. `parameter Real R = 100` becomes `100.0`).

Optimization & Transformation Passes

Once lowered to the arena, equations pass through four high-performance transformations:



1. **Constant Folding (`foldArenaConstants`):** Recursively simplifies compile-time constant arithmetic expressions directly in the arena.
2. **Alias Elimination (`eliminateArenaAliases`):** Detects trivial equality equations ($x = y$) generated by connectors and replaces all references with a single canonical variable handle.
3. **Pantelides Index Reduction:** Differentiates high-index algebraic constraints to reduce the DAE index to 1 or 0 for stable numerical integration.
4. **Tarjan BLT Partitioning:** Constructs the bipartite incidence graph between variables and equations, computes maximum matching, and finds Strongly Connected Components (SCC) using Tarjan's algorithm.

CHAPTER 8

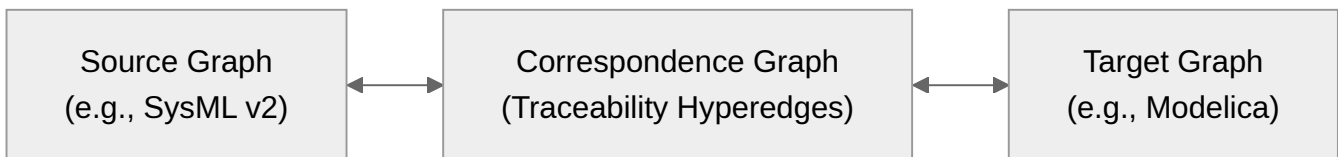
Polyglot Triple Graph Grammars (TGG)

ModelScript provides declarative, ahead-of-time (AOT) compiled **Triple Graph Grammars (TGG)** and Double-Pushout (DPO) algebraic graph rewriting via `@modelscrip/dsl/tgg`.

TGG serves as the foundational synchronization mechanism across the polyglot digital thread, enabling bidirectional transformations between disparate engineering models (such as architectural SysML v2 specifications and 1D Modelica physical simulations).

What is a Triple Graph Grammar?

A Triple Graph Grammar synchronizes three distinct graphs:



- **Source Graph (G_S):** Represents domain concepts in the source language (e.g. SysML v2 Part Definitions, Ports, Requirements).
- **Target Graph (G_T):** Represents concepts in the target language (e.g. Modelica Models, Connectors, Equations).
- **Correspondence Graph (G_C):** Explicitly records semantic trace links between elements in the source and target models.

Rather than writing ad-hoc imperative glue code, engineers write declarative rules specifying how elements correspond.

Declarative TGG Rules

A TGG rule specifies patterns in the source, correspondence, and target graphs simultaneously:

```
import { tggRule } from "@modelscript/dsl/tgg";

export const ComponentToModel = tggRule({
  name: "ComponentToModel",
  source: (s) => ({
    part: s.node("PartUsage", { name: s.var("compName") }),
    port: s.node("PortUsage", { parent: s.nodeRef("part") })
  }),
  target: (t) => ({
    model: t.node("ModelDefinition", { name: t.var("compName") }),
    pin: t.node("PinConnector", { parent: t.nodeRef("model") })
  }),
  correspondence: (c) => ({
    tracePart: c.link("part", "model"),
    tracePort: c.link("port", "pin")
  })
});
```

AOT Compilation to WebAssembly

Unlike interpreted graph transformation engines, `@modelscript/dsl/tgg` compiles rules into deterministic AssemblyScript dispatch routines executing directly in WebAssembly:

- `tgg_forward_dispatch` : Incremental forward propagation from source changes to target models.
- `tgg_backward_dispatch` : Reverse propagation of parameter sizing or component changes back to architectural definitions.
- `tgg_propagate_all_stale` : High-throughput batch reconciliation across dirty workspace subgraphs.

Critical Pair Analysis (CPA)

A notorious problem in graph transformation is rule conflict and non-determinism. ModelScript includes an automated **Critical Pair Analysis (CPA)** verification pass:

1. **Confluence Verification**: Analyzes whether overlapping rule applications could cause divergent target states.
2. **Termination Guarantees**: Proves that cyclic dependencies cannot cause infinite rewrite loops during synchronization.
3. **Completeness Checking**: Flags unmapped source elements that lack corresponding target constructors.

CHAPTER 9

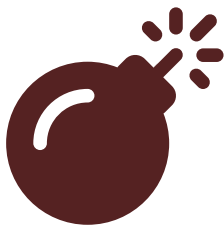
Nelson-Oppen Semantic Theory Coordinator

Cross-domain engineering verification requires proving invariants that span discrete taxonomies, non-linear physical dynamics, spatial CAD envelopes, and numerical constraints simultaneously.

ModelScript solves this through a generalized **Nelson-Oppen Semantic Theory Coordinator** (`@modelscrip/runtime/formal/theory_coordinator`).

Theory Combination Architecture

The Nelson-Oppen framework orchestrates signature-disjoint theory solvers. Each solver operates on its specific domain algebra while communicating variable equalities, numeric interval bounds, and conflict clauses via a shared CDCL(T) coordinator:



Syntax error in text
mermaid version 11.16.1

The 6 Domain Theory Oracles

1. Ontology Domain (`OntologyTheoryOracle`)

- **Underlying Logic:** Description Logic (DL) tableau reasoner.
- **Capabilities:** OWL2 taxonomy reasoning, concept classification, concept subsumption ($C \sqsubseteq D$), and scoped disjointness.
- **Verification Role:** Ensures components conform to standardized ontology definitions (e.g. confirming a redundant actuator cannot share a single power bus).

2. Constraint Domain (`ConstraintTheoryOracle`)

- **Underlying Logic:** DPLL(T) arithmetic solver with non-linear interval arithmetic.
- **Capabilities:** HC4 constraint propagation and box consistency.
- **Verification Role:** Enforces algebraic inequality bounds ($u_{\min} \leq u(t) \leq u_{\max}$) and parameter limits.

3. Abstract Domain (`AbstractDomainOracle`)

- **Underlying Logic:** Abstract interpretation over numerical domains.
- **Capabilities:** Octagon Difference Bound Matrices ($\pm x_i \pm x_j \leq c$) and 4D spatio-temporal bounding boxes.

- **Verification Role:** Efficiently bounds state spaces without expensive simulation.

4. Continuous Safety Domain (`ContinuousSafetyOracle`)

- **Underlying Logic:** Formal reachability analysis and Lyapunov barrier certificates.
- **Capabilities:** Constrained zonotopes, star sets, and Sum-of-Squares (SOS) polynomial optimization.
- **Verification Role:** Proves that continuous ODE/DAE trajectories never enter unsafe sets under uncertain initial conditions.

5. Dynamic Simulation Domain (`DynamicSimulationOracle`)

- **Underlying Logic:** High-order numerical DAE trajectory integration.
- **Capabilities:** Evaluates Signal Temporal Logic (STL) formulas ($\Box_{[0,T]}(\text{speed} < 120)$) and computes quantitative robustness degrees.
- **Verification Role:** Validates complex dynamic behaviors that exceed purely symbolic verification.

6. Spatial Physics Domain (`SpatialPhysicsOracle`)

- **Underlying Logic:** Exact boundary representation (B-Rep) 3D CAD analysis.
- **Capabilities:** Watertight solid boolean collision checks, clearance verification, center of mass, and moment of inertia extraction.
- **Verification Role:** Ensures physical components physically fit inside their geometric enclosures without interference.

Verification Workflow Example

To prove an end-to-end requirement:

```
npx msc verify SystemRequirement model.mo --sysml architecture.sysml --cad chassis.step
```

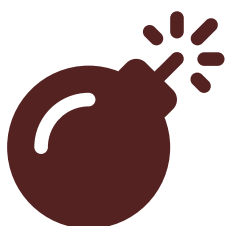
1. **System Architecture** is ingested from SysML v2.
2. **Physical Dynamics** are simulated and bounded from Modelica.
3. **Clearance Limits** are verified against the STEP CAD solid.
4. If a conflict arises, the Theory Coordinator computes an **interpolant lemma** or minimal unsatisfiable core (Unsat Core) explaining the root cause.

CHAPTER 10

Simulation Solvers & Numerical Integrators

ModelScript embeds a multi-tier suite of numerical solvers in `@modelscript/simulate` to support both desktop simulations and massive batched cloud/WebGPU parameter sweeps.

Supported Solvers



Syntax error in text
mermaid version 11.16.1

1. SUNDIALS CVODE & IDA Integrators

For high-precision industrial models, ModelScript integrates Lawrence Livermore National Laboratory's **SUNDIALS** suite compiled to WebAssembly:

- **CVODE**: Solves Initial Value Problems (IVPs) for stiff and non-stiff systems of ordinary differential equations (ODEs). Uses Variable-Order Variable-Step-size Adams-Moulton methods for non-stiff systems and Backward Differentiation Formulas (BDF) for stiff systems.
- **IDA**: Solves Initial Value Problems for systems of differential-algebraic equations (DAEs) of index up to 1:

$$F(t, y, y') = 0$$

Includes dense and sparse linear solvers with adaptive step-size control.

2. WebGPU Batched Parallel Solvers

When performing Monte Carlo uncertainty analysis or hyperparameter sweeps, sequential simulation is a bottleneck.

ModelScript compiles linearized DAE blocks into WebGPU compute shaders (`@modelscript/simulate/gpu`):

- Runs thousands of parameter combinations in parallel on GPU hardware.
- Scales efficiently across client GPUs in browsers and headless datacenter instances.
- Returns statistical distributions, mean trajectories, and variance envelopes directly.

3. Direct Collocation Optimal Control

ModelScript formulates dynamic optimization and optimal control problems directly from Modelica models:

$$\min_{u(t)} \int_0^T L(x(t), u(t), t) dt$$

$$\text{subject to } F(t, x, \dot{x}, u) = 0, \quad u_{\min} \leq u(t) \leq u_{\max}$$

- **Transcription:** Transcribes the continuous system into non-linear programming (NLP) problems using Radau or Lobatto collocation points.
- **Solvers:** Solves the resulting sparse NLP using interior-point optimization techniques.

4. Surrogate Reduced Order Models (ROM)

High-fidelity 3D simulations (such as CFD or FEA) are often too computationally expensive for real-time control loops or web deployment.

ModelScript's surrogate modeling pipeline (`msc surrogate`):

1. **Sampling:** Generates experimental design samples (Latin Hypercube, Sobol sequences).
2. **Execution:** Evaluates high-fidelity runs across parameter spaces.
3. **Training:** Trains compact surrogate models using:
 - Polynomial Chaos Expansion (PCE)
 - Gaussian Process Regression (Kriging)
 - Neural Ordinary Differential Equations (Neural ODEs)
4. **Embedding:** Exports the trained surrogate as a standardized FMI Functional Mock-up Unit (FMU) or standalone WebAssembly module.

PART III

Polyglot Languages & Formats

CHAPTER 11

Polyglot Engineering Languages

ModelScript breaks down domain silos by providing first-class compiler and language server support for nine engineering languages. Every language features a native WebAssembly GLR incremental parser, shared CST arenas, and unified semantic queries.

Language Support Matrix

Language	Directory	Standard / Dialect	Primary Domain	Parser Technology
Modelica	<code>languages/modelica</code>	Modelica 3.x	1D Multi-Domain Physical Simulation	WASM GLR (<code>parser.wasm</code>)
SysML v2	<code>languages/sysml2</code>	SysML v2 / KerML	Systems Architecture & Requirements	WASM GLR (<code>parser.wasm</code>)
STEP	<code>languages/step</code>	ISO 10303-21/203/214/242	3D CAD B-Rep Product Data	WASM GLR (<code>parser.wasm</code>)
OWL2	<code>languages/owl2</code>	OWL 2 Functional-Style	Ontological Knowledge & Taxonomies	WASM GLR (<code>parser.wasm</code>)
CSV	<code>languages/csv</code>	RFC 4180 / Sensor CSV	Telemetry & Calibration Datasets	WASM GLR (<code>parser.wasm</code>)
CFD	<code>languages/cfd</code>	SU2 / OpenFOAM dialects	Fluid Dynamics & Flow Boundaries	WASM GLR (<code>parser.wasm</code>)
FEA	<code>languages/fea</code>	Nastran / Code_Aster	Structural Stress & Finite Elements	WASM GLR (<code>parser.wasm</code>)
OpenSCAD	<code>languages/scad</code>	OpenSCAD CSG	Programmatic 3D Solid Geometry	WASM GLR (<code>parser.wasm</code>)
SSP	<code>languages/ssp</code>	Modelica Assoc. SSP 1.0	Co-Simulation System Packaging	WASM GLR / XML Parser

Unified Multi-Language Indexing

All languages share a unified symbol indexing interface defined in `@modelscrip/dsl`:

- **Flat Symbol Storage:** Every declaration is assigned a unique `symbolId` and stored in a flat table with byte offsets back to source.

- **Cross-Language References:** A SysML v2 component can directly reference a Modelica simulation model or STEP CAD file using URI identifiers.
- **Single Workspace Index:** The Language Server Protocol (LSP) indexes mixed-language workspaces simultaneously, enabling cross-language go-to-definition, hover documentation, and rename refactorings.



Syntax error in text
mermaid version 11.16.1

CHAPTER 12

Modelica Language Support

ModelScript provides full-featured compiler, linter, and simulation capabilities for **Modelica 3.x** under `languages/modelica/`.

The implementation includes complete equation-based modeling, connector semantics, inheritance hierarchies, and rigorous conformance testing against the OpenModelica testsuite.

Flattening Pipeline (`ModelicaFlattener`)

The compiler lowers hierarchical Modelica classes into a flat system of differential-algebraic equations (DAE).

Backend Execution Modes

The flattener (`languages/modelica/src/flattener.ts`) supports four execution backends:

- `"hybrid"` (*default*): High-performance mode coordinating the in-WASM AssemblyScript flattening kernel (`assembly/flattener.ts`) with the TypeScript Salsa query bridge.
- `"wasm"` : Fully in-WASM kernel execution operating exclusively within linear memory buffers.
- `"ts"` : Pure TypeScript fallback backend for debugging and detailed AST introspection.
- `"diff"` : Conformance verification mode that runs both the WASM kernel and TS reference flattener side-by-side to detect drift.

Connector Balancing & Stream Semantics

ModelScript implements strict Modelica connection semantics:

- **Potential Variables** (non-flow): Equality equations generated across all connected pins:

$$v_1 = v_2 = \dots = v_n$$

- **Flow Variables** (e.g. current, flow rate): Conservation equations summing to zero:

$$\sum_{i=1}^n i_k = 0$$

- **Stream Variables**: Upstream operator evaluation and mixing equations for fluid dynamics.

Example Modelica Model

```

model DC_Motor "Permanent magnet DC motor with mechanical inertia"
  // Parameters
  parameter Real R = 1.5 "Armature resistance (Ohm)";
  parameter Real L = 0.05 "Armature inductance (H)";
  parameter Real k = 0.02 "Torque and back-EMF constant";
  parameter Real J = 0.001 "Rotor inertia (kg.m^2)";
  parameter Real b = 0.0001 "Viscous friction coefficient";

  // State variables
  Real i(start = 0, fixed = true) "Armature current (A)";
  Real w(start = 0, fixed = true) "Angular velocity (rad/s)";
  Real phi(start = 0, fixed = true) "Rotor angle (rad)";

  // Inputs
  input Real u "Applied voltage (V)";
equation
  // Electrical equation
  L * der(i) + R * i = u - k * w;

  // Mechanical equation
  J * der(w) + b * w = k * i;

  // Kinematic relation
  der(phi) = w;
end DC_Motor;

```

CLI Compilation & Flattening

Compile and flatten a Modelica class using `msc` :

```

# Flatten to canonical DAE equations
npx msc flatten DC_Motor motor.mo

# Simulate for 10 seconds and output CSV trajectory
npx msc simulate DC_Motor motor.mo --stop-time 10 --output trajectory.csv

```

CHAPTER 13

SysML v2 & KerML Support

ModelScript supports the OMG **Systems Modeling Language v2 (SysML v2)** and the underlying **Kernel Modeling Language (KerML)** under `languages/sysml2/`.

Core Capabilities

- **WASM GLR Parser:** High-throughput parsing of `.sysml` and `.kerm1` files into linear CST nodes.
 - **KerML Standard Library:** Pre-indexed snapshots of foundational mathematical, spatial, and unit definitions (`ScalarValues` , `Quantities` , `ISQ`).
 - **Structural Definitions:** Full AST queries for `part def` , `item def` , `port def` , and hierarchical usage trees (`part :> ...`).
 - **Behavioral Semantics:** State machines (`state` , `transition`), action flows (`action` , `accept`), and calculation blocks.
 - **Requirements Engineering:** `requirement def` modeling with satisfaction links (`satisfy`) and verification cases (`verify`).
-

Example SysML v2 Model

```

package VehicleArchitecture {
  private import ScalarValues::*;
  private import ISQ::*;

  part def BatteryPack {
    attribute capacity : Energy;
    attribute mass : Mass;
    port powerOut : ElectricalPowerPort;
  }

  part def ElectricPowertrain {
    part battery : BatteryPack;
    part inverter : Inverter;
    part motor : TractionMotor;

    connection c1 : PowerConnection
      connect battery.powerOut to inverter.powerIn;
  }

  requirement def MaxSpeedRequirement {
    doc /* Vehicle must achieve 150 km/h top speed under nominal payload. */
    attribute targetSpeed : Speed = 150 [km/h];
  }
}

```

Cross-Domain Verification

SysML v2 requirements are mapped into the Nelson-Oppen Theory Coordinator:

```

# Verify SysML v2 requirements against Modelica physical behavior
npx msc verify MaxSpeedRequirement powertrain.sysml --modelica VehicleDynamics.mo

```

CHAPTER 14

STEP CAD (ISO 10303) Support

ModelScript supports ISO 10303 Product Data Representation and Exchange (**STEP**) under `languages/step/`.

Supported Protocols & Schemas

- **AP203:** Configuration Controlled 3D Designs of Mechanical Parts and Assemblies.
- **AP214:** Core Data for Automotive Mechanical Design Processes.
- **AP242:** Managed Model-Based 3D Engineering (incorporating semantic PMI/GD&T).

Architectural Role: 3D CAD in the Digital Thread

Traditional physical simulation abstracts mechanical bodies into point masses or 1D rotational inertias. ModelScript connects real 3D CAD geometry directly into physical simulations and verification loops:

1. **Mass Properties Extraction:** Automatically computes volume, center of mass, and the full 3×3 moment of inertia tensor ($J_{xx}, J_{yy}, J_{zz}, J_{xy}, J_{xz}, J_{yz}$) directly from STEP solids.
2. **Clearance & Enclosure Checking:** Feeds watertight solid boundaries to the `SpatialPhysicsOracle` to guarantee that components do not collide under thermal expansion or structural deflection.
3. **Dynamic Visual Binding:** Modelica simulation variables can be bound to STEP geometry via `DynamicSelect` annotations for 3D visualization.

Ingesting STEP Models

```
# Ingest and verify mass properties of a mechanical chassis
npx msc csg evaluate chassis.step --mass-properties

# Render STEP assembly hierarchy and component trees
npx msc render chassis.step --format svg
```

CHAPTER 15

OWL2 Ontologies & Taxonomies

ModelScript supports the W3C **OWL 2 Web Ontology Language (OWL2)** Functional-Style Syntax under `languages/owl2/`.

Why Ontologies in Engineering Compilers?

Complex engineering enterprises face severe vocabulary mismatch: electrical, software, and mechanical teams often use different names for the same subsystem, or contradictory definitions for redundancy and safety classifications.

By incorporating OWL2 directly into the compiler:

- Engineering taxonomies are formally parsed into the `SymbolIndex`.
- Description Logic (DL) reasoning verifies concept subsumption and disjointness at compile time.
- System architectures are checked against domain ontologies before simulation begins.

Example OWL2 Ontology

```
Prefix(=<http://modelscript.org/ontologies/aerospace#>)
Prefix(owl:=<http://www.w3.org/2002/07/owl#>)

Ontology(<http://modelscript.org/ontologies/aerospace>
  Declaration(Class(:FlightCriticalSystem))
  Declaration(Class(:Actuator))
  Declaration(Class(:DualRedundantActuator))

  SubClassOf(:DualRedundantActuator :Actuator)
  SubClassOf(:DualRedundantActuator :FlightCriticalSystem)

  DisjointClasses(:SinglePointFailureComponent :FlightCriticalSystem)
)
```

Reasoning with `ontologyTheoryOracle`

The `ontologyTheoryOracle` in `@modelscript/runtime/formal` connects OWL2 axioms into Nelson-Open theory combination:

- **Subsumption:** Verifies whether a declared component instance A satisfies concept requirement C ($A \sqsubseteq C$).

- **Disjointness:** Identifies architectural violations where a single component mistakenly inherits from mutually disjoint classes.

CHAPTER 16

CSV Telemetry & Tabular Data

ModelScript provides high-throughput tabular parsing and telemetry validation under `Languages/csv/`.

Core Capabilities

- **High-Throughput SIMD / WASM Parser:** Streams megabyte-scale telemetry files into linear memory buffers with zero intermediate string allocation.
- **Typed Column Inference:** Automatically detects IEEE 754 floating-point, integer, boolean, timestamp, and string column schemas.
- **Trajectory Interpolation:** Binds continuous Modelica simulation time (t) to discrete sampled measurement tables using cubic spline or linear interpolation.
- **Parameter Calibration Datasets:** Supplies reference trajectories to non-linear least-squares optimization loops for model parameter tuning.

Example Usage in Modelica

Tabular CSV data can be referenced directly in Modelica simulations:

```
model SensorValidation
  Modelica.Blocks.Sources.CombiTimeTable testBenchData(
    tableOnFile = true,
    tableName = "tab1",
    fileName = "testbench_run_042.csv"
  );
  Real measuredRPM;
  Real simulatedRPM;
equation
  measuredRPM = testBenchData.y[1];
  // Calibration error formulation
  error = simulatedRPM - measuredRPM;
end SensorValidation;
```

CHAPTER 17

Computational Fluid Dynamics (CFD) Support

ModelScript supports Computational Fluid Dynamics boundary conditions, mesh specifications, and configuration dialects under `languages/cfd/`.

Dialects Supported

- **SU2:** Configuration files (`.cfg`) for aeronautical shape optimization and compressible/incompressible flow analysis.
 - **OpenFOAM:** Dictionary files (`fvSchemes` , `fvSolution` , boundary condition files).
-

Capabilities in the Digital Thread

1. **Boundary Condition Ingestion:** Parses velocity inlets, pressure outlets, wall boundary roughness, and turbulence models ($k-\epsilon$, SST $k-\omega$).
2. **Coupled Co-Simulation (1D/3D):** Couples 1D Modelica hydraulic/pneumatic piping systems with 3D continuum CFD flow domains via co-simulation interfaces.
3. **Aerodynamic Coefficient Extraction:** Extracts lift (C_L), drag (C_D), and pitching moment (C_M) polar surfaces to generate reduced-order aerodynamic tables for fast flight simulation.

CHAPTER 18

Finite Element Analysis (FEA) Support

ModelScript supports Finite Element Analysis configurations, structural mesh metadata, and load case specifications under `languages/fea/`.

Core Capabilities

- **Material Property Definitions:** Ingests isotropic, orthotropic, and anisotropic material tensors (Young's modulus E , Poisson's ratio ν , yield strength σ_y).
- **Boundary & Load Conditions:** Parses structural fixtures (fixed, pinned, roller) and applied force/moment distributions.
- **Stress & Deflection Fields:** Extracts nodal von Mises stress contours, principal stresses, and displacement vectors.
- **Thermal-Structural Coupling:** Translates temperature distributions computed by Modelica thermal circuits into thermal strain load cases for FEA solvers.

CHAPTER 19

OpenSCAD Constructive Solid Geometry (CSG)

ModelScript provides programmatic 3D CAD modeling through OpenSCAD language support under `languages/scad/`.

Core Capabilities

- **CSG AST Parsing & Evaluation:** Parses OpenSCAD scripts (`.scad`) and evaluates Constructive Solid Geometry trees into geometric primitives and boolean operations.
- **Boolean Transformations:** Supports 3D `union()` , `difference()` , and `intersection()` operations.
- **Parametric Extraction:** Automatically extracts parametric dimensions, wall thicknesses, and hole diameters for sensitivity analysis and optimization.
- **Direct Export:** Evaluates OpenSCAD models into STEP B-Rep solids, STL triangle meshes, or SVG cross-sections.

Example OpenSCAD Model

```
// Parametric mounting bracket
width = 50;
length = 80;
thickness = 6;
hole_radius = 4;

difference() {
  cube([length, width, thickness], center = true);
  translate([length/3, 0, 0])
    cylinder(r = hole_radius, h = thickness + 2, center = true);
  translate([-length/3, 0, 0])
    cylinder(r = hole_radius, h = thickness + 2, center = true);
}
```

Evaluating with `msc`

```
# Evaluate OpenSCAD and generate a watertight STEP solid
npx msc csg evaluate bracket.scad --output bracket.step
```

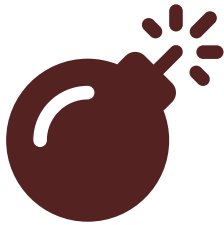
CHAPTER 20

System Structure & Parameterization (SSP)

ModelScript provides full support for the Modelica Association standard **System Structure and Parameterization (SSP 1.0)** under `languages/ssp/` and `@modelscrip/exchange`.

What is SSP?

SSP defines a standardized, tool-independent format for packaging complete simulation systems composed of multiple interconnected Functional Mock-up Units (FMUs), parameters, and signal dictionaries.



Syntax error in text
mermaid version 11.16.1

Core Capabilities

- **SSP Archive Packaging & Unpacking:** Creates and unpacks standardized `.ssp` ZIP archives containing multiple FMUs and metadata.
- **Topological Connection Resolution:** Validates input-output signal compatibility across connected FMUs.
- **Unit Conversion & Signal Mappings:** Evaluates linear scale and offset transformations across connected ports ($y = a \cdot u + b$).
- **Co-Simulation Orchestration:** Drives co-simulation master algorithms across all bundled FMUs with adaptive communication step sizes.

PART IV

Comprehensive Algorithms Catalog

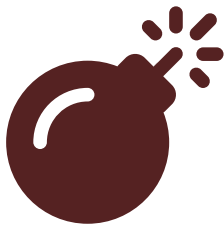
CHAPTER 21

ModelScript Algorithms Catalog

ModelScript is powered by a comprehensive foundation of peer-reviewed mathematical, compiler, and formal verification algorithms. Rather than relying on opaque external black boxes, ModelScript implements these algorithms natively in WebAssembly and TypeScript, with inline citations, architectural rationales, and explicit tracking of upstream inputs and downstream consumers.

Computational Map

The algorithms catalog spans seven core domains across the compilation, simulation, and verification pipelines:



Syntax error in text
mermaid version 11.16.1

Algorithm Directory

Category	Primary Algorithms	Implementation Package
Structural DAE	Pantelides Algorithm, Tarjan BLT, Cellier-Elmqvist Tearing	@modelscript/runtime
Acausal Balancing	Acausal Potential Equalization, Kirchhoff Flows, Stream Mixing	@modelscript/modelica
Autodiff & Relaxations	Dual Numbers, AdTape, CPR Graph Coloring, McCormick, Affine	@modelscript/runtime
Numerical Solvers	Tsit5, Rodas-4P, TR-BDF2, SDE, BVP	@modelscript/simulate
Optimization	L-BFGS-B, CMA-ES, Differential Evolution, PSO, NSGA-II	@modelscript/simulate
SMT & Verification	Nelson-Oppen, CDCL SAT, Craig Interpolation, HC4, STL, Zonotopes	@modelscript/runtime/formal
Model Checking	Spacer CHC PDR & IC3 Engine	@modelscript/runtime/formal
Abstract Interpretation	WTO Fixpoint, TVPI, Octagon DBM, Array Segments	@modelscript/runtime/formal
Graph Rewriting	Triple Graph Grammars, CPA, WCOJ, E-Graphs, GSS, Salsa	@modelscript/dsl
Linear Memory Structures	Roaring Bitmaps, Front-Coded Dictionary, Paged B+ Tree	@modelscript/runtime
Scientific Computing	Sparse Cholesky, Sparse LU, Householder QR, Gröbner, POD, Buckingham Pi	@modelscript/runtime & @modelscript/simulate
SDP, BVP & CAD	Primal-Dual SDP, Multiple Shooting BVP, CSG & CFD Patches	@modelscript/runtime , @modelscript/simulate & @modelscript/cad
Diagram Layout	Tamassia TSM, Port ILP Solver, Dogleg Channel Router	@modelscript/diagram

CHAPTER 22

Pantelides Structural Index Reduction

Implementation: `PantelidesEngine` in `packages/runtime/src/wasm/structural/pantelides.ts`

Academic Citations

- **Pantelides, C. C. (1988).** *"The consistent initialization of differential-algebraic systems."* **SIAM Journal on Scientific and Statistical Computing**, 9(2), pp. 213–231.
DOI: [10.1137/0909014](https://doi.org/10.1137/0909014)
- **Mattsson, S. E., & Söderlind, G. (1993).** *"Index reduction in differential-algebraic equations using dummy derivatives."* **SIAM Journal on Scientific Computing**, 14(3), pp. 677–692.
DOI: [10.1137/0914043](https://doi.org/10.1137/0914043)

ModelScript Architectural Rationale

Object-oriented multi-domain physical models (such as constrained multi-body mechanics, closed kinematic linkages, and incompressible hydraulic loops) inherently generate high-index Differential-Algebraic Equations (DAEs) where the differential index satisfies $\text{ind} \geq 2$:

$$F(t, x, \dot{x}, y) = 0$$

Standard numerical integrators (including BDF, Radau, and Runge-Kutta methods) become severely ill-conditioned or fail to converge on systems with differential index greater than 1.

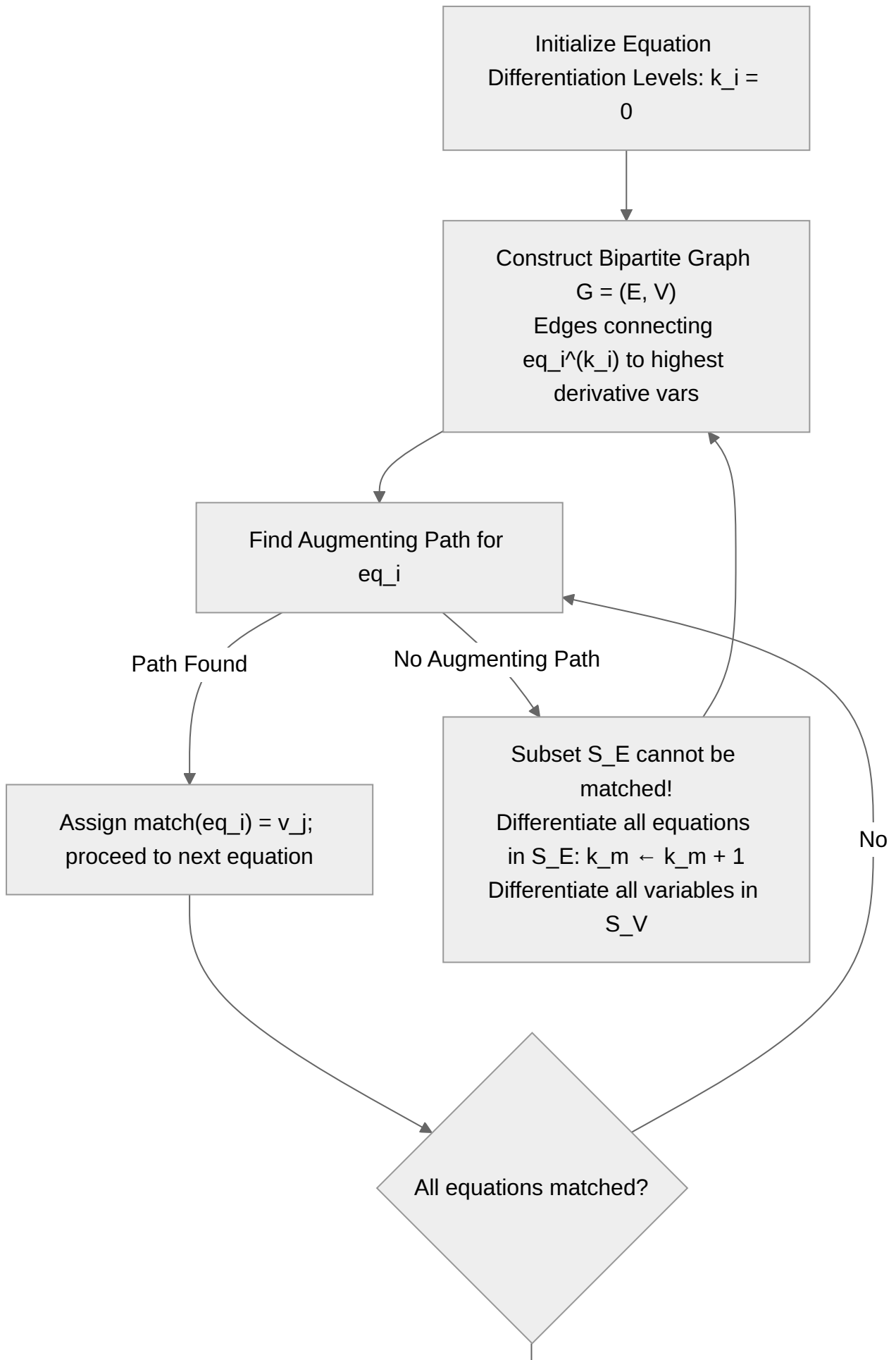
Pantelides' algorithm identifies minimally singular subsets of equations and differentiates them analytically with respect to time until an index-1 or index-0 formulation is attained. To prevent over-determination and numerical drift along algebraic constraint manifolds, the Mattsson-Söderlind dummy derivative method dynamically replaces redundant derivative states with algebraic placeholders.

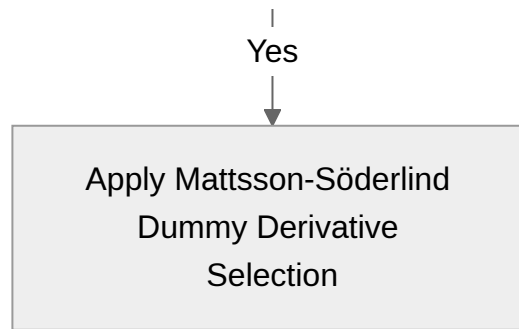
ModelScript Modifications & WASM Architecture

1. **In-WASM Linear Memory Execution:** Operates as an unmanaged WebAssembly kernel (`@unmanaged`) with zero garbage collection allocations.
2. **Direct Arena Symbolic Differentiation:** Derivatives of equations ($\frac{d^k f}{dt^k} = 0$) are differentiated directly on `DAEBuilders` linear expression nodes in memory.
3. **Flat Chunked Arrays:** Equation dependency incidence pointers, differentiation levels, and matching arrays are stored in flat `ChunkedInt32Array` buffers.

4. **Augmenting Path Bipartite Matching:** Alternating path searches run directly over highest-derivative variables, seamlessly transferring matching data to the downstream `BLtEngine`.
-

Algorithm Description & Mathematics





Mathematical Steps

1. **Variable Classification:** Identify highest-order derivatives $\dot{x}_j$ and purely algebraic variables y_k .
 2. **Alternating Augmenting Paths:** For each equation $f_i = 0$, attempt to find an augmenting path in the bipartite incidence graph to an unmatched highest-order derivative.
 3. **Index Reduction by Differentiation:** If no augmenting path exists, the reachable set of equations S_E is structurally singular. Increment the differentiation index $k_m \leftarrow k_m + 1$ for all $f_m \in S_E$, symbolically compute $\frac{df_m}{dt}$, and repeat the matching.
 4. **Dummy Derivatives:** For each differentiated state variable where both x_j and $\dot{x}_j$ appear in algebraic constraints, select a dummy algebraic variable $\{j, \text{dummy}\}x_j, \text{dummy}$ to replace $\dot{x}_j$, preserving degrees of freedom.
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Ingests the flat DAE representation from `DAEBuilder` produced by `ModelicaFlattener`.
 - Uses `cst` linear nodes for symbolic expressions.
- **Downstream Consumers:**
 - Feeds the index-reduced DAE directly into `BltEngine` for block lower triangular partitioning.
 - Determines consistent initial conditions for `SUNDIALS IDA` and `TR-BDF2`.

CHAPTER 23

Tarjan Block Lower Triangular (BLT) Transformation

Implementation: `BltEngine` in `packages/runtime/src/wasm/structural/blt.ts`

Academic Citations

- **Tarjan, R. E. (1972).** "Depth-first search and linear graph algorithms." *SIAM Journal on Computing*, 1(2), pp. 146–160.
DOI: [10.1137/0201010](https://doi.org/10.1137/0201010) (Strongly Connected Components)
- **Hopcroft, J. E., & Karp, R. M. (1973).** "An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs." *SIAM Journal on Computing*, 2(4), pp. 225–231.
DOI: [10.1137/0202019](https://doi.org/10.1137/0202019) (Bipartite Matching)
- **Dulmage, A. L., & Mendelsohn, N. S. (1958).** "Coverings of bipartite graphs." *Canadian Journal of Mathematics*, 10, pp. 517–534.
DOI: [10.4153/CJM-1958-052-0](https://doi.org/10.4153/CJM-1958-052-0)
- **Cellier, F. E., & Kofman, E. (2006).** *Continuous System Simulation*. Springer. ISBN: [978-0-387-26102-7](https://doi.org/10.1007/978-0-387-26102-7)

ModelScript Architectural Rationale

Flattening industrial physical systems results in coupled systems comprising thousands of equations and variables. Solving a monolithic non-linear algebraic system of size N using general Newton-Raphson methods incurs a cubic computational penalty per step:

$$O(N^3)$$

The Block Lower Triangular (BLT) transformation permutes the equation and variable ordering so that the incidence matrix assumes a lower triangular block diagonal structure:

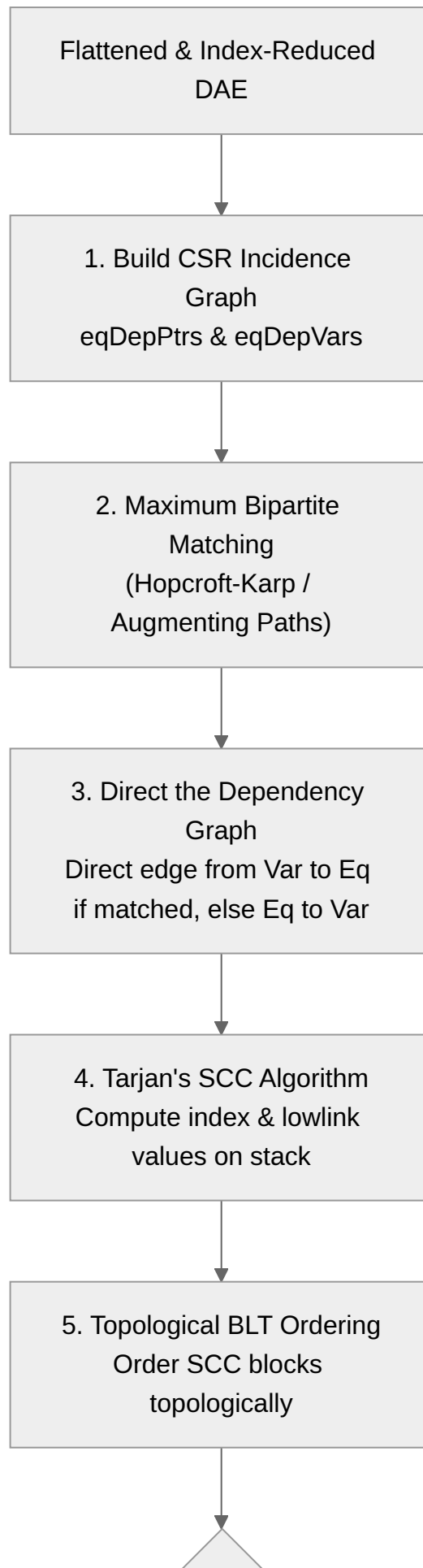
$$\begin{pmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mm} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

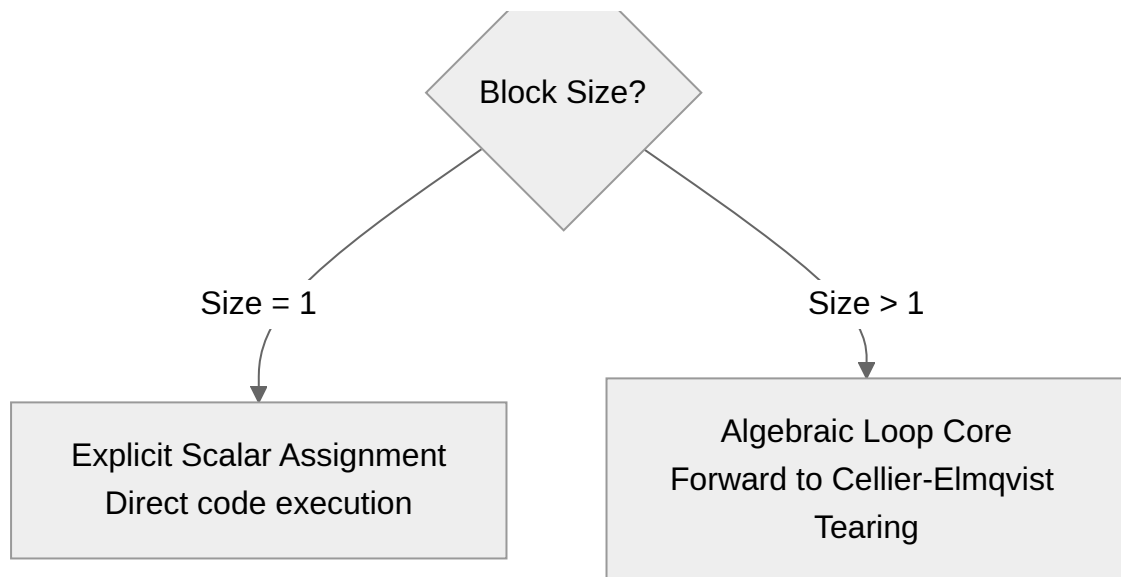
Most blocks A_{ij} are trivial 1×1 scalar explicit assignments ($x_i = f(\dots)$), while only non-linear algebraic loops become coupled sub-blocks of size $k \ll N$.

ModelScript Modifications & WASM Architecture

1. **Compressed Sparse Row (CSR) in Linear Memory:** Bipartite incidence graphs are stored in contiguous `ChunkedInt32Array` buffers (`eqDepPtrs` , `eqDepVars`).
 2. **Zero-Allocation Tarjan Traversal:** Low-link values, depth indices, and traversal stacks operate strictly on 32-bit integer offsets in WebAssembly memory without object references.
 3. **Integrated Maximum Matching:** Hopcroft-Karp augmenting path search provides consistent equation-to-variable assignments for directed SCC analysis.
 4. **Direct Bridge to Tearing:** SCC blocks with dimension > 1 pass directly to the in-WASM `TornBlock` engine.
-

Algorithm Description & Pipeline Flow





Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Ingests index-reduced DAE systems from `PantelidesEngine`.
- **Downstream Consumers:**
 - Trivial 1×1 blocks are lowered directly into sequence evaluation bytecode.
 - Coupled blocks (size > 1) are sent to `TornBlock` for algebraic loop tearing.
 - Drives step evaluations in `SUNDIALS CVODE/IDA` and `Tsit5`.

CHAPTER 24

Cellier-Elmqvist Algebraic Loop Tearing

Implementation: `TornBlock` in `packages/runtime/src/wasm/structural/tearing.ts`

Academic Citations

- **Cellier, F. E. (1979).** *Combined Continuous/Discrete System Simulation by Use of Digital Computers: Techniques and Tools.*
PhD thesis, ETH Zurich.
DOI: [10.3929/ethz-a-000179979](https://doi.org/10.3929/ethz-a-000179979)
- **Elmqvist, H., & Otter, M. (1994).** "Methods for Tearing Systems of Equations in Object-Oriented Modeling." *Proceedings of the European Simulation Multiconference (ESM'94)*, pp. 326–332.
- **Mahana, P. N., & Eagan, J. P. (1975).** "A tearing algorithm for large systems of equations." *IEEE Transactions on Circuits and Systems*.

ModelScript Architectural Rationale

Even after BLT partitioning, physical systems containing closed kinematic loops, electrical bridge circuits, or multi-phase fluid balances yield coupled Strongly Connected Components of size $N > 1$.

Solving an N -dimensional non-linear system via Newton-Raphson requires inverting an $N \times N$ Jacobian matrix at every iteration:

$$O(N^3)$$

The Cellier-Elmqvist Minimum Degree Tearing algorithm identifies a minimal set of k **tearing variables** ($k \ll N$) such that assuming known values for these k variables breaks all cyclic dependencies in the block. The remaining $N - k$ inner variables can then be evaluated sequentially via a causal explicit forward substitution chain:

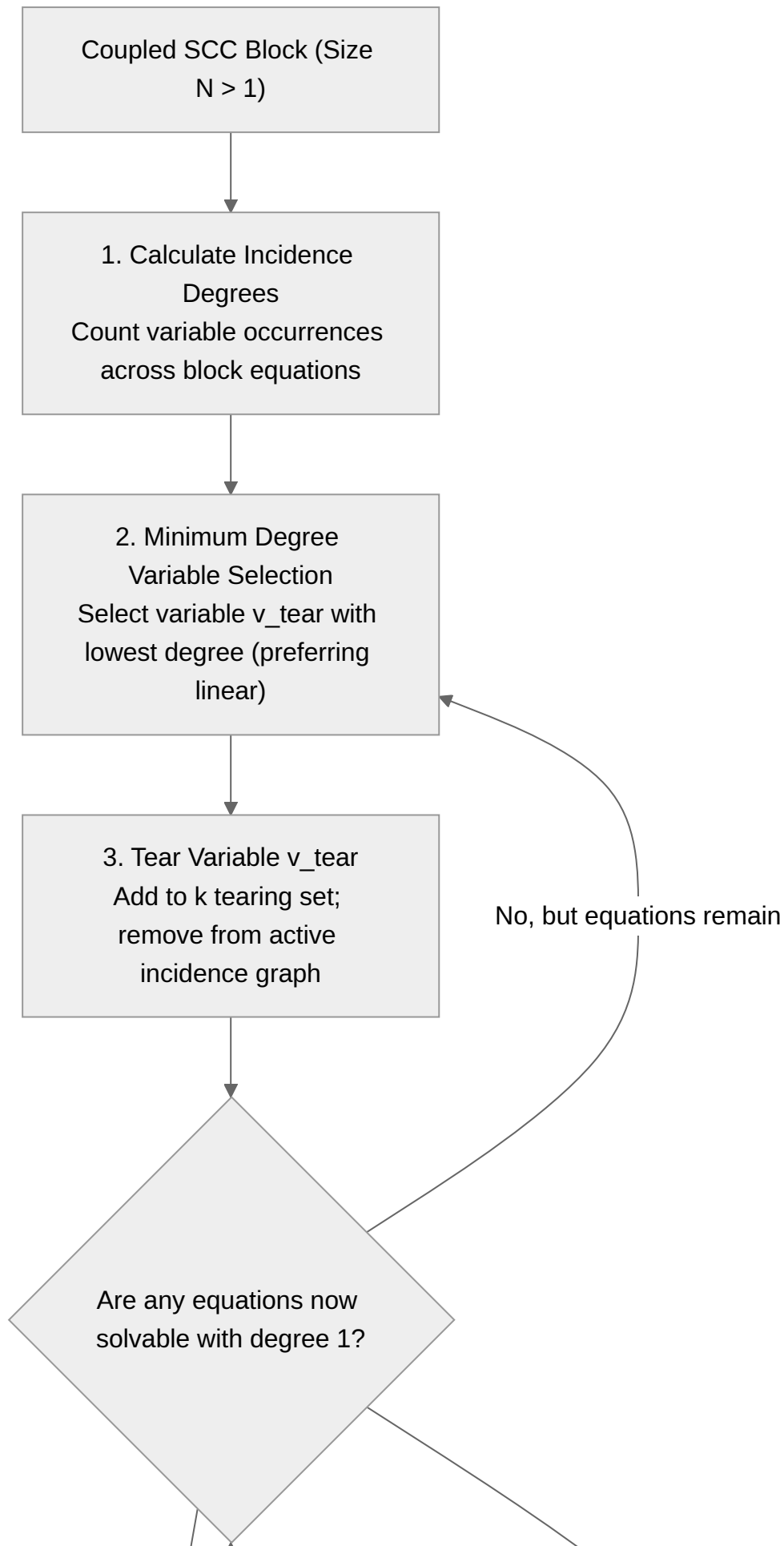
$$\begin{aligned} x_{\text{inner}} &= g(x_{\text{tear}}, u) \quad (\text{explicit chain}) \\ R(x_{\text{tear}}) &= f_{\text{residual}}(x_{\text{tear}}, x_{\text{inner}}) = 0 \quad (\text{small } k \times k \text{ solve}) \end{aligned}$$

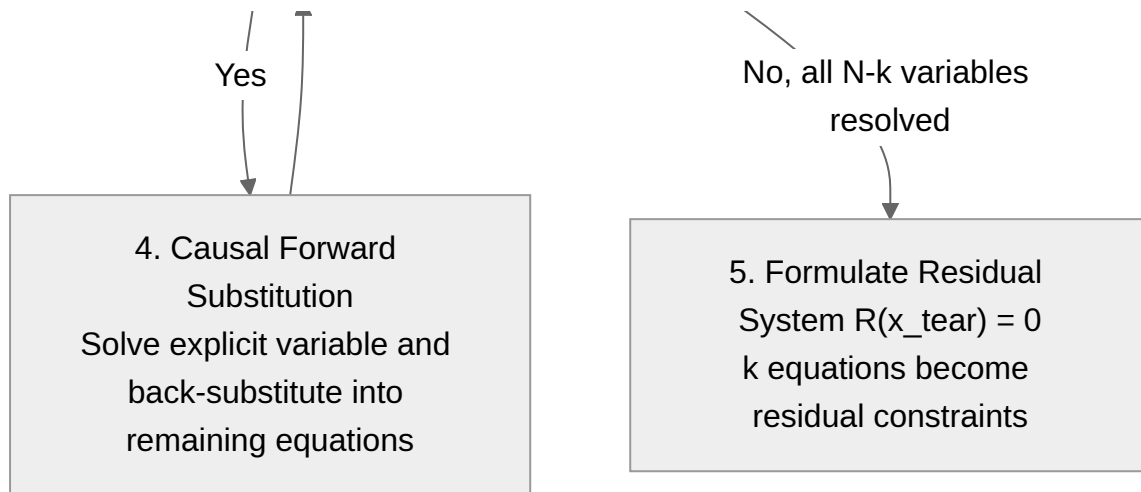
The non-linear iterative solver is restricted to the tiny $k \times k$ residual system, accelerating simulation speed by orders of magnitude.

ModelScript Modifications & WASM Architecture

1. **Unmanaged WebAssembly Kernel:** Implemented as `@unmanaged` classes directly in linear memory with zero garbage collection pauses.
 2. **Dynamic Minimum Degree Heuristic:** Evaluates vertex degrees in the block incidence graph in memory, prioritizing linear tearing variables over non-linear variables to avoid non-linear solving where possible.
 3. **Partitioned In-Memory Structures:** Splits the block into:
 - k tearing variables (`tearVarIndices`)
 - $N - k$ explicit causal equations (`innerEqIndices`)
 - k residual constraint equations (`residualEqIndices`)
 4. **Direct Matrix Integration:** Emits residual Jacobians directly to the in-WASM LU factorizer (`luFactor` , `luSolve`).
-

Algorithm Steps





Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Receives coupled SCC blocks of size > 1 from `BltEngine`.
- **Downstream Consumers:**
 - Feeds explicit forward chains and $k \times k$ residual systems to the in-WASM Newton-Raphson solver and `Sparse LU Solver`.
 - Evaluated inside simulation step loops (`SUNDIALS` , `TR-BDF2` , `Tsit5`).

CHAPTER 25

Acausal Connector Balancing & Stream Mixing

Implementation: `ConnectionSetResolver` & `IntUnionFind` in `languages/modelica/src/connections.ts`

Academic Citations

Acausal Physical Modeling & Connection Sets

- **Mattsson, S. E., Elmqvist, H., & Otter, M. (1998).** *"Physical system modeling with Modelica."* **Control Engineering Practice**, 6(4), pp. 501–510.
DOI: [10.1016/S0967-0661\(98\)00047-1](https://doi.org/10.1016/S0967-0661(98)00047-1)
- **Tarjan, R. E. (1975).** *"Efficiency of a good but not linear set union algorithm."* **Journal of the ACM**, 22(2), pp. 215–225.
DOI: [10.1145/321879.321884](https://doi.org/10.1145/321879.321884) (*Disjoint-Set Union-Find*)

Stream Connectors & Convective Transport

- **Franke, R., Casella, F., Otter, M., Sielemann, M., Elmqvist, H., Mattsson, S. E., & Olsson, H. (2009).** *"Stream Connectors - An Extension of Modelica for Device-Oriented Modeling of Convective Transport."* **Proceedings of the 7th Modelica Conference**, pp. 116–123.
DOI: [10.3384/ecp09430076](https://doi.org/10.3384/ecp09430076)

Expandable Connectors & Multi-Domain Bus Modeling

- **Otter, M., Elmqvist, H., & Mattsson, S. E. (2005).** *"Object-oriented modeling of bus systems with Modelica."* **Proceedings of the 4th Modelica Conference**, pp. 295–308.
[Paper PDF](#)

ModelScript Architectural Rationale

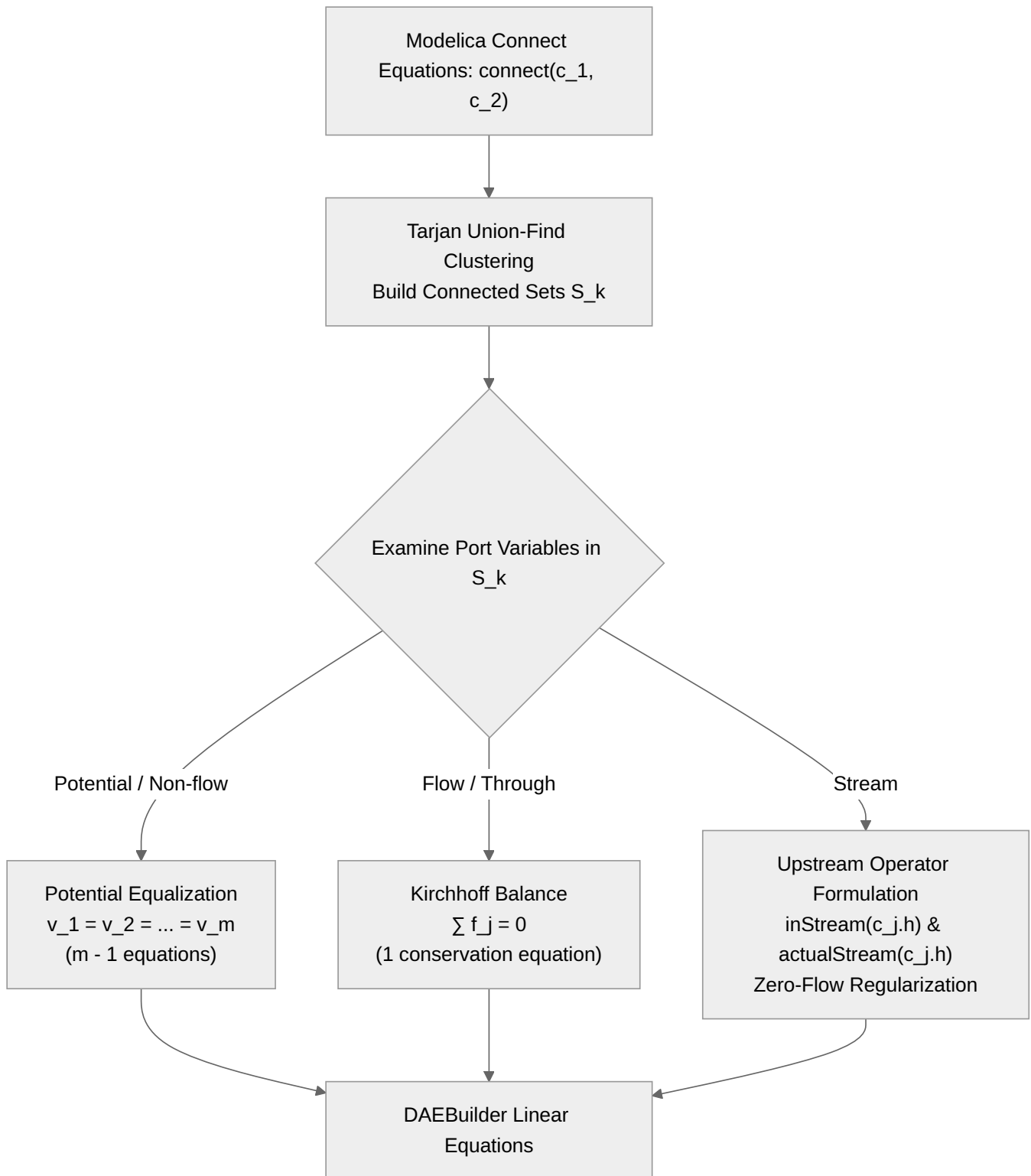
Unlike classical block diagrams (such as Simulink) where connections represent directional causal signal flows ($y = f(u)$), ModelScript models **acausal physical interaction** across multi-energy domains:

- **Electrical:** Voltage v (potential) and Current i (flow).
- **Hydraulic & Pneumatic:** Pressure p (potential) and Volume/Mass flow rate q (flow).
- **Translational Mechanics:** Position/Velocity s, v (potential) and Force f (flow).
- **Rotational Mechanics:** Angle/Angular velocity ϕ, ω (potential) and Torque τ (flow).
- **Thermal & Fluid Transport:** Temperature/Enthalpy h (stream) and Heat/Mass flow $\dot{m}$ (flow).

In acausal modeling, connection statements `connect(a, b)` do not dictate computational causality. The compiler must automatically generate:

1. **Potential Equality Equations:** Enforcing that all across-variables in a connection set share an identical value.
 2. **Kirchhoff Flow Equations:** Enforcing conservation of mass, charge, or energy such that all through-variables sum to zero at the junction.
 3. **Convective Stream Transport Equations:** Resolving thermodynamic mixing when fluid direction reverses ($m_j \geq 0$) without introducing singular divisions near zero mass flow.
-

Balancing Algorithm & Mathematical Formulation



1. Disjoint Connection Set Resolution (Union-Find)

The set of all declared `connect(c_a, c_b)` statements forms an undirected graph. ModelScript partitions all connector references into disjoint equivalence classes using an integer array-based Union-Find with path compression and rank heuristics:

$$\alpha(N) \leq 4 \quad \text{for all physical models}$$

For each connection set $S = \{c_1, c_2, \dots, c_m\}$ of size m :

2. Potential Variable Equalization

For every non-flow, non-stream variable v declared inside the connector type:

$$v(c_1) = v(c_2) = \dots = v(c_m)$$

This yields exactly $m - 1$ scalar equations. One variable is chosen as the canonical representative, eliminating $m - 1$ redundant states during alias elimination.

3. Kirchhoff Flow Variable Balancing

For every `flow` variable f declared inside the connector type:

$$\sum_{j=1}^m f(c_j) = 0$$

This yields exactly 1 scalar conservation equation per connection set. If an inside-outside connector hierarchy boundary is crossed, the outer flow sign is inverted to preserve directional conservation:

$$f_{\text{outer}} = \sum_{j \in \text{inner}} f(c_j)$$

4. Stream Variable Convective Transport

For fluid ports, transport enthalpy depends on flow direction:

$$h_{\text{transported}} = \begin{cases} h_{\text{upstream}} & \text{if } \dot{m} > 0 \\ h_{\text{downstream}} & \text{if } \dot{m} < 0 \end{cases}$$

Direct formulation introduces a discontinuous switch that causes Newton solvers to oscillate around $\dot{m} = 0$. ModelScript lowers stream variables using Franke's regularized `inStream()` and `actualStream()` operators:

$$\text{inStream}(c_i.h) = \frac{\sum_{j \in \mathbb{I}, \dot{m}_j > 0} \dot{m}_j \cdot c_j.h}{\sum_{j \in \mathbb{I}, \dot{m}_j > 0} \dot{m}_j}$$

Near zero flow ($\|\dot{m}_j\| < \epsilon$), smooth polynomial regularization is applied to maintain C^1 -continuity of the residual Jacobian.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
IntUnionFind	AST <code>connect</code> clauses from <code>Cst</code>	Connection set partition groups
Potential Equalizer	Variable causality & variability from <code>QueryEngine</code>	Alias elimination (<code>eliminateArenaAliases</code>)
Flow Balancer	Connector definitions	<code>DAEBuild</code> er linear equations
Stream Formulator	Fluid connector declarations	<code>PantelidesEngine</code> , Nonlinear tearing loops

CHAPTER 26

Automatic Differentiation & Relaxations

Implementation:

- Forward Mode: `Dual` in `packages/runtime/src/wasm/autodiff/dual.ts`
- Reverse Mode: `AdTape` in `packages/runtime/src/wasm/autodiff/tape.ts`
- Sparse Jacobian Coloring: `CCSMatrix` & `ColoringEngine` in `packages/runtime/src/wasm/autodiff/coloring.ts`
- McCormick Envelopes: `McCormickTuple` in `packages/runtime/src/wasm/autodiff/mccormick.ts`
- Affine Arithmetic: `AffineForm` in `packages/runtime/src/wasm/autodiff/affine.ts`

Academic Citations

Forward-Mode Dual Numbers

- **Clifford, W. K. (1873).** *"Preliminary sketch of biquaternions."* **Proceedings of the London Mathematical Society**, 4, pp. 381–395.
DOI: [10.1112/plms/s1-4.1.381](https://doi.org/10.1112/plms/s1-4.1.381)
- **Study, E. (1903).** *Geometrie der Dynamen: Die Zusammensetzung von Kräften und verwandte Gegenstände der Geometrie.* B.G. Teubner.
- **Rall, L. B. (1981).** *Automatic Differentiation: Techniques and Applications.* **Lecture Notes in Computer Science**, 120, Springer.
DOI: [10.1007/3-540-10861-0](https://doi.org/10.1007/3-540-10861-0)

Reverse-Mode Adjoint Accumulation (Wengert List)

- **Wengert, R. E. (1964).** *"A simple automatic derivative evaluation program."* **Communications of the ACM**, 7(8), pp. 463–464.
DOI: [10.1145/355586.364791](https://doi.org/10.1145/355586.364791)
- **Linnainmaa, S. (1976).** *"Taylor expansion of the accumulated rounding error."* **BIT Numerical Mathematics**, 16(2), pp. 146–160.
DOI: [10.1007/BF01931367](https://doi.org/10.1007/BF01931367)
- **Griewank, A., & Walther, A. (2008).** *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (2nd ed.). SIAM.
ISBN: [978-0-898716-59-7](https://www.isbn-international.org/product/9780898716597)

Distance-2 Graph Coloring for Sparse Jacobians

- **Curtis, A. R., Powell, M. J. D., & Reid, J. K. (1974).** *"On the estimation of sparse Jacobian matrices."* **Journal of the Institute of Mathematics and Its Applications**, 13(1), pp. 117–119.
DOI: [10.1093/imamat/13.1.117](https://doi.org/10.1093/imamat/13.1.117) (CPR Algorithm)

- **Coleman, T. F., & Moré, J. J. (1983).** "Estimation of sparse Jacobian matrices and graph coloring problems." *SIAM Journal on Numerical Analysis*, 20(1), pp. 187–209.
DOI: [10.1137/0720013](https://doi.org/10.1137/0720013)
- **Gebremedhin, A. H., Manne, F., & Pothén, A. (2005).** "What color is your Jacobian? Graph coloring for computing derivatives." *SIAM Review*, 47(4), pp. 629–705.
DOI: [10.1137/S0036144504446096](https://doi.org/10.1137/S0036144504446096)

McCormick Relaxations & Affine Arithmetic

- **McCormick, G. P. (1976).** "Computability of global solutions to factorable nonconvex programs: Part I—Convex underestimating problems." *Mathematical Programming*, 10(1), pp. 147–175.
DOI: [10.1007/BF01580665](https://doi.org/10.1007/BF01580665)
- **Mitsos, A., Chachuat, B., & Barton, P. I. (2009).** "McCormick-based relaxations of algorithms." *SIAM Journal on Optimization*, 20(2), pp. 573–601.
DOI: [10.1137/080717341](https://doi.org/10.1137/080717341)
- **Tsoukalas, A., & Mitsos, A. (2014).** "Multivariate McCormick relaxations." *Journal of Global Optimization*, 59(2), pp. 633–662.
DOI: [10.1007/s10898-014-0169-z](https://doi.org/10.1007/s10898-014-0169-z)
- **de Figueiredo, L. H., & Stolfi, J. (2004).** "Affine arithmetic: concepts and applications." *Numerical Algorithms*, 37(1), pp. 147–158.
DOI: [10.1023/B:NUMA.0000049462.70970.b6](https://doi.org/10.1023/B:NUMA.0000049462.70970.b6)

ModelScript Architectural Rationale

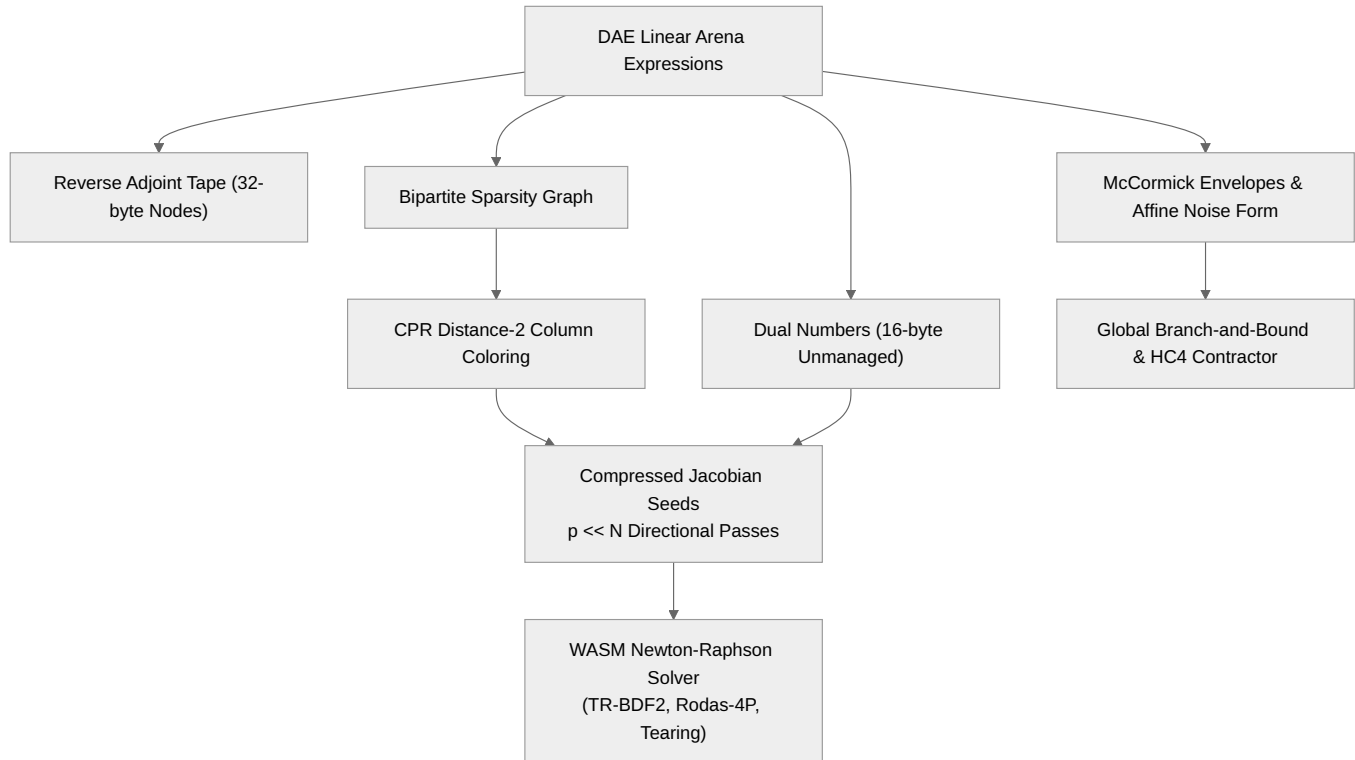
Numerical finite-difference approximations ($\frac{f(x+h)-f(x)}{h}$) suffer from a fundamental trade-off: large step sizes h cause truncation errors, while small step sizes cause catastrophic floating-point cancellation.

ModelScript implements an integrated suite of **exact differentiation and continuous bounding engines** inside WebAssembly linear memory:

1. **Forward-Mode Dual Numbers:** Computes exact machine-precision directional derivatives $\nabla f(x) \cdot v$ concurrently with primal function evaluation for Newton steps in algebraic loops and stiff ODE/DAE integrators without allocating expression trees.
2. **Reverse-Mode Computational Tape:** Evaluates scalar objective gradients $\nabla f(x) \in \mathbb{R}^N$ with cost proportional to $O(1)$ evaluations of f , enabling gradient-based parameter estimation and surrogate neural network training.
3. **CPR Distance-2 Graph Coloring:** For large-scale DAE systems ($N = 10,000+$ equations), evaluating the full $N \times N$ Jacobian naively requires N forward passes. CPR distance-2 coloring partitions structural columns into $p \ll N$ non-interfering groups (typically $p \leq 15$ for sparse physical networks), reducing Jacobian assembly time by orders of magnitude.
4. **McCormick Envelopes & Affine Arithmetic:** Classical interval arithmetic suffers from the **dependency problem**: evaluating $x - x$ for $x \in [0, 1]$ yields $[-1, 1]$ instead of 0, causing rapid exponential bounding blowup. Affine arithmetic tracks first-order noise symbols ($\epsilon_i \in [-1, 1]$) across state equations, while

McCormick relaxations compute rigorous convex lower bounds (f^R) and concave upper bounds (f^{AR}) for non-convex global optimization.

ModelScript Modifications & WASM Architecture



1. Zero-GC Dual Number Arithmetic

Represented as a 16-byte unmanaged struct (`@unmanaged`) in WASM memory:

```
@unmanaged
export class Dual {
  val: f64; // Primal evaluation f(x)
  dot: f64; // Directional derivative f'(x) * v
}
```

All elementary operators (+, −, ×, /) and transcendental functions (sin, cos, exp, log, pow) are inlined without intermediate object allocation.

2. Linear Memory Computational Tape

The reverse-mode tape records operations into a flat 32-byte stride unmanaged array:

Stride: [op: u32, left: u32, right: u32, aux: u32, valLo: u32, valHi: u32, gradLo: u32, gradHi: u32]

During the reverse pass, adjoints are accumulated in topological order directly in linear memory:

$$\bar{x}_j \leftarrow \bar{x}_j + \bar{x}_i \cdot \frac{\partial f_i}{\partial x_j}$$

3. CPR Distance-2 Coloring Algorithm

Two columns j and k in the Jacobian J are **structurally orthogonal** if they do not share any non-zero row entry:

$$\text{Row}(j) \cap \text{Row}(k) = \emptyset$$

The graph coloring algorithm: 1. Constructs the bipartite graph $B = (V_R, V_C, E)$ where edge $(i, j) \in E \iff \frac{\partial f_i}{\partial x_j} \neq 0$. 2. Colors columns V_C such that any pair of columns connected by a path of length 2 receives different colors. 3. Groups columns of color $c \in \{1, \dots, p\}$ into a perturbation vector $d^{(c)} = \sum_{j \in \text{Color}(c)} e_j$. 4. Computes the entire compressed Jacobian in exactly p directional forward-mode passes:

$$J \cdot d^{(c)} = \sum_{j \in \text{Color}(c)} J_{*,j}$$

4. McCormick Relaxations

For factorable bilinear products $z = x \cdot y$ over bounds $x \in [x^L, x^U]$ and $y \in [y^L, y^U]$, the convex underestimator z^{cv} and concave overestimator z^{cc} are:

$$z^{cv} = \max(x^L y + x y^L - x^L y^L, x^U y + x y^U - x^U y^U)$$

$$z^{cc} = \min(x^U y + x y^L - x^U y^L, x^L y + x y^U - x^L y^U)$$

ModelScript evaluates McCormick tuples $[z^{cv}, z^{cc}, \nabla z^{cv}, \nabla z^{cc}]$ directly on WASM arrays.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
Dual Numbers	DAEBuilder AST, Variable seed vector	Algebraic loop tearing, Newton solvers, sensitivity analysis
AdTape	DAEBuilder objective expressions	L-BFGS-B optimizer, surrogate loss backprop
CPR Coloring	Bipartite incidence matrix from BltEngine	Sparse Jacobian generation, SUNDIALS/TR-BDF2 linear solves
McCormick & Affine	Bound intervals from QueryEngine	HC4 contractor, Nelson-Oppen theory coordinator, SOS bounds

CHAPTER 27

Numerical ODE, DAE & Co-Simulation Solvers

Implementation: `packages/simulate/src/solvers/` and `packages/runtime/src/solvers/`

1. Tsitouras 5(4) Adaptive Runge-Kutta (Tsit5)

Academic Citations

- **Tsitouras, Ch. (2011).** *"Runge-Kutta pairs of order 5 (4) satisfying only the first column simplifying assumption."*
Computers & Mathematics with Applications, 62(2), pp. 770–775.
DOI: [10.1016/j.camwa.2011.06.002](https://doi.org/10.1016/j.camwa.2011.06.002)
- **Rackauckas, C., & Nie, Q. (2017).** *"DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia."*
Journal of Open Research Software, 5(1), 15. DOI: [10.5334/jors.151](https://doi.org/10.5334/jors.151)

ModelScript Rationale & Modifications

For non-stiff systems (e.g. mechanical multi-body dynamics, kinematics, flight control), high-order adaptive explicit Runge-Kutta pairs offer peak throughput without Newton-Raphson linear algebra costs. Tsit5 demonstrates significantly lower error coefficients and wider stability regions than classical Dormand-Prince (DOPRI5).

- **Modifications:** Employs the First-Same-As-Last (FSAL) optimization eliminating stage 1 evaluation in successive steps; embeds a 5th-order continuous dense output polynomial for sub-interval event zero-crossing localization.

2. TR-BDF2 Composite Stiff Solver

Academic Citations

- **Bank, R. E., Coughran, W. M., et al. (1985).** *"Transient simulation of silicon devices and circuits."*
IEEE Transactions on CAD of Integrated Circuits, 4(4), pp. 436–446.
DOI: [10.1109/TCAD.1985.1270142](https://doi.org/10.1109/TCAD.1985.1270142)
- **Hosea, M. E., & Shampine, L. F. (1996).** *"Analysis and implementation of TR-BDF2."*
Applied Numerical Mathematics, 20(1-2), pp. 21–37.
DOI: [10.1016/0168-9274\(95\)00115-8](https://doi.org/10.1016/0168-9274(95)00115-8)

ModelScript Rationale & Modifications

Cyber-physical models featuring frequent hybrid discrete events (switches, impacts, hysteresis) severely penalize multistep BDF solvers (which must drop to order 1 after every event). TR-BDF2 is an L-stable one-

step composite method pairing a trapezoidal half-step with a BDF2 completion step:

- **Modifications:** Shares the diagonal Jacobian coefficient $d = 1 - 1/\sqrt{2}$ across both internal stages, requiring only **one** sparse LU factorization per time step; connects directly with ModelScript's in-WASM sparse matrix kernel (`sparse_lu.ts`).

3. RODAS-4P Linearly Implicit Rosenbrock Solver

Academic Citations

- **Steinebach, G. (1995).** *"Order-reduction of ROW-methods for DAEs and method of lines applications."* TH Darmstadt Preprint 1742.
- **Hairer, E., & Wanner, G. (1996).** *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems.* Springer-Verlag. Section IV.8.

ModelScript Rationale & Modifications

Solves stiff index-1 DAEs and method-of-lines continuum discretizations without iterative non-linear Newton solves. It solves only a sequence of linear systems sharing the same system matrix $I - \gamma h J$.

- **Modifications:** Optimized for in-WASM sparse Jacobian caching; utilizes automatic differentiation coloring (`coloring.ts`) for fast Jacobian updates.

4. Itô Stochastic Differential Equations (SDE) Solver

Academic Citations

- **Rößler, A. (2010).** *"Runge-Kutta methods for Itô stochastic differential equations with additive noise."* SIAM Journal on Numerical Analysis, 48(3), pp. 922–952. DOI: [10.1137/09076636X](https://doi.org/10.1137/09076636X)
- **Kloeden, P. E., & Platen, E. (1992).** *Numerical Solution of Stochastic Differential Equations.* Springer. DOI: [10.1007/978-3-662-12616-5](https://doi.org/10.1007/978-3-662-12616-5)

ModelScript Rationale & Modifications

Physical systems operate under thermal noise, turbulence, and sensor fluctuations:

$$dx = f(t, x) dt + g(t, x) dW_t$$

- ****Modifications**:** Employs an in-WASM `Xoshiro256pp` PRNG with Box-Muller transformation; provides both fixed-step Euler-Maruyama (strong order 0.5) and adaptive Rößler SRIW1 (strong order 1.5) integrators for Monte Carlo uncertainty rollouts.

5. Co-Simulation Master Orchestrator

Academic Citations

- **Bastian, J., et al. (2011).** *"Master for Co-Simulation Using FMI."* **Proceedings of the 8th International Modelica Conference**, pp. 115–120. DOI: [10.3384/ecp11063115](https://doi.org/10.3384/ecp11063115)
- **Kübler, R., & Schiehlen, W. (2000).** *"Two methods of simulator coupling."* **Mathematical and Computer Modelling of Dynamical Systems**, 6(2), pp. 93–113. DOI: [10.1076/1387-3954\(200006\)6:2;1-M;FT093](https://doi.org/10.1076/1387-3954(200006)6:2;1-M;FT093)

ModelScript Rationale & Modifications

Coordinates multi-rate, multi-vendor FMI Functional Mock-up Units (FMUs) and SSP topologies. Implements Richardson extrapolation error estimation to dynamically adapt the inter-simulator communication step size H_{comm} , preventing divergence while maximizing simulation speed.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - DAE blocks partitioned and torn by `BltEngine` and `TornBlock`.
 - Sparse Jacobians computed via `Dual Numbers & Graph Coloring`.
- **Downstream Consumers:**
 - Powers `msc simulate`, `msc optimize`, and the Web IDE charting canvas.
 - Feeds state trajectories into `Signal Temporal Logic (STL) Monitor` and formal verification oracles.

CHAPTER 28

Mathematical Optimization & Calibration Solvers

Implementation: `packages/simulate/src/optimizer/solvers/`

1. Limited-Memory BFGS with Bounds (L-BFGS-B)

Academic Citations

- Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995). "A limited memory algorithm for bound constrained optimization." *SIAM Journal on Scientific Computing*, 16(5), pp. 1190–1208. DOI: [10.1137/0916069](https://doi.org/10.1137/0916069)
- Zhu, C., Byrd, R. H., Lu, P., & Nocedal, J. (1997). "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization." *ACM Transactions on Mathematical Software*, 23(4), pp. 550–560. DOI: [10.1145/279232.279236](https://doi.org/10.1145/279232.279236)

ModelScript Rationale & Modifications

Solves large-scale bound-constrained non-linear minimization:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to } l \leq x \leq u$$

Crucial for parameter identification and direct collocation optimal control where thousands of discretized states and control bounds must be optimized. - ****Modifications****: Employs compact representation of the inverse Hessian approximation H_k using history buffers $m \in [5, 20]$ in typed arrays; Cauchy point computation with piecewise linear path search; exact line search with strong Wolfe conditions.

2. Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

Academic Citations

- Hansen, N., & Ostermeier, A. (2001). "Completely derandomized self-adaptation in evolution strategies." *Evolutionary Computation*, 9(2), pp. 159–195. DOI: [10.1162/106365601750190398](https://doi.org/10.1162/106365601750190398)
- Hansen, N. (2016). "The CMA Evolution Strategy: A Tutorial." *arXiv preprint: arXiv:1604.00772*.

ModelScript Rationale & Modifications

Derivative-free black-box optimization on rugged, non-convex, non-separable objective functions (e.g. tuning PID controller gains over simulations with discontinuous switching events).

- **Modifications:** Tracks full covariance matrix $C \in \mathbb{R}^{n \times n}$ updates with rank- μ and rank-one update mechanisms; vectorized population sampling using WebAssembly-accelerated Cholesky decomposition; step-size control using cumulative path lengths (CSA).
-

3. Differential Evolution (DE)

Academic Citations

- **Storn, R., & Price, K. (1997).** *"Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces."*
Journal of Global Optimization, 11(4), pp. 341–359. DOI: [10.1023/A:1008202821328](https://doi.org/10.1023/A:1008202821328)

ModelScript Rationale & Modifications

Stochastic population-based global search algorithm utilizing vector differences for directional perturbations. Highly effective for parameter estimation against experimental CSV test bench data.

- **Modifications:** Supports `rand/1/bin`, `best/1/bin`, and `current-to-best/1` mutation strategies with adaptive crossover probability (CR) and scaling factor (F).
-

4. Particle Swarm Optimization (PSO)

Academic Citations

- **Kennedy, J., & Eberhart, R. (1995).** *"Particle swarm optimization."*
Proceedings of IEEE ICNN'95, Vol. 4, pp. 1942–1948. DOI: [10.1109/ICNN.1995.488968](https://doi.org/10.1109/ICNN.1995.488968)
- **Shi, Y., & Eberhart, R. C. (1998).** *"A modified particle swarm optimizer."*
IEEE Congress on Evolutionary Computation, pp. 69–73.

ModelScript Rationale & Modifications

Simulates social swarm dynamics for continuous global search:

$$v_i(t+1) = wv_i(t) + c_1r_1(p_i - x_i) + c_2r_2(g - x_i)$$

- ****Modifications****: Implements constriction factors and dynamically declining inertia weights $w(t)$ to transition smoothly from global exploration to local exploitation.

5. Non-Dominated Sorting Genetic Algorithm II (NSGA-II)

Academic Citations

- **Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002).** *"A fast and elitist multiobjective genetic algorithm: NSGA-II."*
IEEE Transactions on Evolutionary Computation, 6(2), pp. 182–197. DOI: [10.1109/4235.996017](https://doi.org/10.1109/4235.996017)

ModelScript Rationale & Modifications

Solves multi-objective engineering trade-offs (e.g. simultaneously minimizing electric vehicle energy consumption while maximizing battery longevity and top speed).

- **Modifications:** Fast non-dominated sorting ($O(MN^2)$ complexity) paired with crowding-distance assignment in continuous parameter spaces; outputs full Pareto fronts directly.
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Trajectory residuals evaluated by `Simulation Solvers` against `CSV Datasets`.
 - Collocation constraint formulations from `msc optimize`.
 - **Downstream Consumers:**
 - Powers `msc falsify` adversarial requirement falsification across SysML v2 / Modelica / CAD.
 - Powers `msc cosim optimize` for black-box multi-FMU tuning across SSP boundaries.
 - Powers `ModelicaCalibrator` with `hybrid-cmaes-lm` (CMA-ES global exploration + Levenberg-Marquardt quadratic polishing).
 - Powers `TradeStudyEngine.evolve` for generative multi-objective Pareto design trade studies.
-

Programmatic TypeScript API

```

import {
  cmaesSolve,
  deSolve,
  psoSolve,
  nsga2Solve,
  type BlackBoxProblem,
} from "@modelscrip/simulate/optimizer";

// 1. Single-Objective Black-Box Optimization (CMA-ES, DE, PSO)
const problem: BlackBoxProblem = {
  dimension: 2,
  bounds: {
    min: new Float64Array([-5, -5]),
    max: new Float64Array([5, 5]),
  },
  fitness: (x) => Math.pow(1 - x[0]!, 2) + 100 * Math.pow(x[1]! - x[0]! * x[0]!, 2),
};

const result = await cmaesSolve(problem, { maxGenerations: 100 });
console.log("Optimum:", result.bestSolution, "Cost:", result.bestFitness);

// 2. Hybrid Global-Local Model Calibration
import { ModelicaCalibrator } from "@modelscrip/simulate/optimizer";

const calibrator = new ModelicaCalibrator(arena, sim, {
  parameters: ["k", "c"],
  parameterBounds: new Map([["k", { min: 1, max: 30 }], ["c", { min: 0.1, max: 10 }]]),
  measurements,
  method: "hybrid-cmaes-lm", // Global CMA-ES basin exploration + Levenberg-Marquardt refinement
});
const calResult = calibrator.calibrate();

```

CLI Command Examples

```

# 1. Adversarial requirement falsification with CMA-ES
msc falsify --formula "always[0,10] (stress <= 180)" --algorithm cma-es --population 20

# 2. Black-box co-simulation parameter tuning with Differential Evolution
msc cosim optimize ssp-archive.ssp --params "kp:0.1:10.0,kd:0.01:1.0" --objective "error_integral"

```

CHAPTER 29

Nelson-Oppen Theory Combination & SMT Solvers

Implementation: `packages/runtime/src/formal/`

1. Nelson-Oppen Cooperation & DPLL(T)

Academic Citations

- **Nelson, G., & Oppen, D. C. (1979).** *"Simplification by cooperating decision procedures."* **ACM Transactions on Programming Languages and Systems (TOPLAS)**, 1(2), pp. 245–257. DOI: [10.1145/357073.357079](https://doi.org/10.1145/357073.357079)
- **Nieuwenhuis, R., Oliveras, A., & Tinelli, C. (2006).** *"Solving SAT and SAT Modulo Theories: From an abstract Davis--Putnam--Logemann--Loveland procedure to DPLL(T)."* **Journal of the ACM**, 53(6), pp. 937–977. DOI: [10.1217856.1217859](https://doi.org/10.1217856.1217859)

ModelScript Rationale & Modifications

Proves cross-domain assertions spanning discrete architectural modes, differential equations, interval constraints, and 3D CAD collisions.

- **Modifications:** Extended beyond traditional software theories (uninterpreted functions, bit-vectors) to **12 engineering domains** including OWL2 Description Logic, CAD B-Rep spatial physics, and DAE continuous trajectories; combines learned conflict clause propagation with Salsa query invalidation.

2. CDCL SAT Solver with VSIDS

Academic Citations

- **Silva, J. P. M., & Sakallah, K. A. (1999).** *"GRASP: A search algorithm for propositional satisfiability."* **IEEE Transactions on Computers**, 48(5), pp. 506–521. DOI: [10.1109/12.769433](https://doi.org/10.1109/12.769433)
- **Moskewicz, M. W., et al. (2001).** *"Chaff: Accelerating SAT design verification."* **ACM/IEEE Design Automation Conference (DAC)**, pp. 530–535. DOI: [10.1145/378239.379017](https://doi.org/10.1145/378239.379017)

ModelScript Rationale & Modifications

Solves propositional satisfiability for discrete mode selection, architectural state machines, and CDCL(T) theory case splitting.

- **Modifications:** Two-watched-literals data structure with contiguous integer arrays; Variable State Independent Decaying Sum (VSIDS) decision heuristic; 1-UIP (First Unique Implication Point) conflict analysis; Luby-sequence restart schedule.

3. Craig Interpolation

Academic Citations

- **Craig, W. (1957).** *"Linear reasoning. A new form of the Herbrand-Gentzen theorem."* **Journal of Symbolic Logic**, 22(3), pp. 250–268. DOI: [10.2307/2963593](https://doi.org/10.2307/2963593)
- **McMillan, K. L. (2003).** *"Interpolation and SAT-based model checking."* **Computer Aided Verification (CAV 2003)**, LNCS 2725, pp. 1–13. DOI: [10.1007/978-3-540-45069-6_1](https://doi.org/10.1007/978-3-540-45069-6_1)
- **Pudlák, P. (1997).** *"Lower bounds for resolution and cutting planes proofs and monotone computations."* **Journal of Symbolic Logic**, 62(3), pp. 981–998. DOI: [10.2307/2275583](https://doi.org/10.2307/2275583)

ModelScript Rationale & Modifications

Given unsatisfiable mutually contradictory formulas $A \wedge B \models \perp$, Craig interpolation synthesizes an interpolant I such that:

$$A \Rightarrow I \quad \text{and} \quad I \wedge B \models \perp$$

where $\text{vars}(I) \subseteq \text{vars}(A) \cap \text{vars}(B)$. - **Modifications**: Employs Pudlák-McMillan resolution graph traversal to generate certified interpolants explaining cross-domain requirement violations without revealing proprietary subsystem equations.

4. HC4 Constraint Contractor

Academic Citations

- **Benhamou, F., Goualard, F., Granvilliers, L., & Puget, J. F. (1999).** *"Revising Hull and Box Consistency."* **International Conference on Logic Programming (ICLP'99)**, pp. 230–244.

ModelScript Rationale & Modifications

Evaluates non-linear algebraic constraints over interval domains without discretization grids:

- **Modifications**: Two-phase HC4Revise operator: bottom-up forward evaluation of interval bounds followed by top-down backward narrowing; eliminates physically impossible state regions before numerical simulation.

5. Signal Temporal Logic (STL) Quantitative Monitor

Academic Citations

- **Maler, O., & Nickovic, D. (2004).** *"Monitoring Temporal Properties of Continuous Signals."* **FORMATS/FTRTFT 2004**, LNCS 3253, pp. 152–166. DOI: [10.1007/978-3-540-30206-3_12](https://doi.org/10.1007/978-3-540-30206-3_12)
- **Donzé, A., & Maler, O. (2010).** *"Robust Satisfaction of Temporal Logic over Real-Valued Signals."* **FORMATS 2010**, LNCS 6246, pp. 92–106. DOI: [10.1007/978-3-642-15297-9_9](https://doi.org/10.1007/978-3-642-15297-9_9)

ModelScript Rationale & Modifications

Monitors simulation trajectories against temporal specifications (e.g. "overshoot must remain below 5% for at least 2 seconds after step input"):

$$\rho(\phi, s, t) \in \mathbb{R}$$

Computes real-valued **quantitative robustness** indicating how far a trajectory is from violating safety boundaries.

6. Constrained Zonotopes

Academic Citations

- **Scott, J. K., Raimondo, D. M., et al. (2016).** *"Constrained zonotopes: A new tool for set-based estimation and fault detection."*
Automatica, 69, pp. 126–136. DOI: [10.1016/j.automatica.2016.02.036](https://doi.org/10.1016/j.automatica.2016.02.036)

ModelScript Rationale & Modifications

Set-based reachability analysis under bounded initial states and parameter uncertainty. Closed under linear transformations, Minkowski addition, and hyperplanar intersection without facet explosion.

- **Modifications:** Implemented as in-WASM linear algebraic routines utilizing singular value decomposition (SVD) for order reduction.
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Requirements ingested from `SysML v2`.
 - Trajectories simulated by `Numerical Solvers`.
 - Spatial boundaries provided by `STEP CAD`.
- **Downstream Consumers:**
 - Powers `msc verify`, generates compliance certificates, and exports mathematical proofs in `LaTeX format`.

CHAPTER 30

Constrained Horn Clauses (Spacer) & IC3/PDR

Implementation: `packages/runtime/src/formal/chc/` and `packages/runtime/src/formal/ic3_engine.ts`

1. Spacer CHC Engine (Generalized PDR)

Academic Citations

- Komuravelli, A., Gurfinkel, A., & Chaki, S. (2014). "SMT-Based Model Checking for Recursive Programs." **Computer Aided Verification (CAV 2014)**, LNCS 8559, pp. 17–34.
DOI: [10.1007/978-3-319-08867-9_2](https://doi.org/10.1007/978-3-319-08867-9_2)
- Hoder, K., & Bjørner, N. (2012). "Generalized Property Directed Reachability." **Theory and Applications of Satisfiability Testing (SAT 2012)**, LNCS 7317, pp. 157–171.
DOI: [10.1007/978-3-642-31612-8_13](https://doi.org/10.1007/978-3-642-31612-8_13)

ModelScript Rationale & Modifications

Constrained Horn Clauses (CHCs) represent systems of logical relations of the form:

$$\forall X. \phi(X) \wedge P_1(X) \wedge \dots \wedge P_k(X) \implies H(X)$$

Spacer solves CHCs by combining inductive generalization, Craig interpolation, and under-approximated reachability trees without bounding the unrolling depth. In ModelScript, Spacer verifies infinite-state state machine transitions and recursive cyber-physical control architectures. - **Modifications**: Directly integrates with ModelScript's in-WASM CDCL SAT core and Craig interpolator; implements specialized heuristics for linear real arithmetic over cyber-physical invariants.

2. IC3 / Property Directed Reachability (PDR)

Academic Citations

- Bradley, A. R. (2011). "SAT-based model checking without unrolling." **VMCAI 2011**, LNCS 6538, pp. 70–87.
DOI: [10.1007/978-3-642-18275-4_7](https://doi.org/10.1007/978-3-642-18275-4_7)
- Een, N., Mishchenko, A., & Brayton, R. (2011). "Efficient implementation of property directed reachability." **Formal Methods in Computer-Aided Design (FMCAD 2011)**, pp. 125–134.

ModelScript Rationale & Modifications

Proves whether safety property P holds across all reachable states of a discrete or hybrid transition system without unrolling the transition relation $T(s, s')$ to a fixed horizon k .

- **Modifications:** Maintains inductive clause frames $F_0, F_1, \dots, F_k$ where F_i over-approximates states reachable in up to i steps; uses counterexample-guided abstraction refinement (CEGAR) to block Proof Obligations (POBs); propagates clauses forward to find an inductive invariant ($F_i \equiv F_{i+1}$) proving unbounded safety.
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - State machine models from `SysML v2`.
 - Mode switching and discrete `when / if` clauses lowered from `Modelica`.
- **Downstream Consumers:**
 - Verifies architectural safety contracts in `msc verify`.
 - Feeds lemmas into `Nelson-Open Coordinator`.

CHAPTER 31

Abstract Interpretation & Polyhedral Domains

Implementation: `packages/runtime/src/formal/abstract_interpretation/`

1. Bourdoncle Weak Topological Ordering (WTO) Fixpoint Solver

Academic Citations

- **Bourdoncle, F. (1993).** *"Efficient chaotic iteration strategies with widenings."* **Formal Methods in Programming and Their Applications**, LNCS 735, pp. 128–141. DOI: [10.1007/BFb0039704](https://doi.org/10.1007/BFb0039704)
- **Cousot, P., & Cousot, R. (1977).** *"Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints."* **ACM POPL'77**, pp. 238–252. DOI: [10.1145/512950.512973](https://doi.org/10.1145/512950.512973)

ModelScript Rationale & Modifications

Computes sound abstract over-approximations of state variables across complex nested loops and recursion without simulating unbounded trajectories.

- **Modifications:** Implements Bourdoncle's recursive hierarchical WTO decomposition to identify minimal loop heads; accelerates convergence using delayed widening (∇) followed by monotonic narrowing (Δ).

2. Two Variables Per Inequality (TVPI) Polyhedral Domain

Academic Citations

- **Simon, A., King, A., & Howe, J. M. (2002).** *"Two Variables per Inequality as an Abstract Domain."* **LOPSTR 2002**, LNCS 2664, pp. 71–89. DOI: [10.1007/3-540-36388-2_7](https://doi.org/10.1007/3-540-36388-2_7)
- **Nelson, G. (1978).** *"An $n^{O(\log n)}$ algorithm for the two-variable-per-inequality integer programming problem."* **Technical Report**, Stanford University.

ModelScript Rationale & Modifications

Captures planar relational linear invariants of the form:

$$ax_i + bx_j \leq c \quad (a, b, c \in \mathbb{Q})$$

Much more expressive than simple intervals or octagons, allowing ModelScript to prove physical conservation laws (e.g. $p_1 + p_2 = \text{const}$) without the exponential vertex-facet explosion of general convex

polyhedra. - **Modifications**: Planar Graham-scan convex hull algorithms in WebAssembly typed memory; relational projection routines optimized for physical parameter spaces.

3. Octagon Difference Bound Matrices (DBM) Domain

Academic Citations

- **Miné, A. (2006).** *"The octagon abstract domain."* **Higher-Order and Symbolic Computation**, 19(1), pp. 31–100.
DOI: [10.1007/s10990-006-8609-1](https://doi.org/10.1007/s10990-006-8609-1)

ModelScript Rationale & Modifications

Restricts linear relationships to constraints of the form:

$$\pm x_i \pm x_j \leq c$$

Achieves cubic time complexity $O(n^3)$ via Floyd-Warshall shortest path algorithms over $2n \times 2n$ potential graphs. - **Modifications**: Direct integration into the `@modelscript/dsl` type inference system and `AbstractDomainOracle` in the Nelson-Open coordinator.

4. Array Segment Partitioning

Academic Citations

- **Cousot, P., Cousot, R., & Logozzo, F. (2011).** *"A Parametric Segmentation Functor for Fully Automatic and Scalable Array Content Analysis."* **ACM POPL'11**, pp. 505–518. DOI: [10.1145/1926385.1926444](https://doi.org/10.1145/1926385.1926444)

ModelScript Rationale & Modifications

Verifies large array equations and indexed multi-dimensional components in Modelica without unrolling every array index into separate scalar variables. Partitions index sets into contiguous segments whose bounds and elements share uniform properties.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Control-flow graphs and equation sets from `ModelicaFlattener`.
- **Downstream Consumers:**
 - Feeds numeric intervals into `HC4 Contractor` and `Nelson-Open Coordinator`.
 - Eliminates out-of-bounds array access and division-by-zero compiler warnings at compile time.

CHAPTER 32

Graph Rewriting, Pattern Matching & Metaprogramming

Implementation: `packages/dsl/src/codegen/rewriting/` and `packages/dsl/src/codegen/runtime/`

1. Triple Graph Grammars (TGG) & Algebraic DPO Rewriting

Academic Citations

- **Schürr, A. (1994).** *"Specification of graph translators with triple graph grammars."* **Graph-Theoretic Concepts in Computer Science (WG '94)**, LNCS 903, pp. 151–163. DOI: [10.1007/3-540-59071-4_45](https://doi.org/10.1007/3-540-59071-4_45)
- **Greenyer, J., & Kindler, E. (2010).** *"Comparing operational execution strategies for triple graph grammars."* **Software & Systems Modeling**, 9(4), pp. 441–469. DOI: [10.1007/s10270-009-0140-2](https://doi.org/10.1007/s10270-009-0140-2)

ModelScript Rationale & Modifications

Synchronizes multi-domain engineering languages (e.g. Modelica 1D physics, SysML v2 requirements, STEP CAD geometry) declaratively without fragile point-to-point imperatively paired translators.

- **Modifications:** AOT-compiles declarative TGG rule sets into an in-WASM AssemblyScript dispatch kernel (`tgg_forward_dispatch`, `tgg_backward_dispatch`); tracks bidirectional correspondences in linear CST memory; achieves $O(\Delta N)$ incremental propagation driven by Salsa invalidations.

2. Critical Pair Analysis (CPA)

Academic Citations

- **Knuth, D. E., & Bendix, P. B. (1970).** *"Simple word problems in universal algebras."* **Computational Problems in Abstract Algebra**, pp. 263–297. Pergamon Press.
- **Ehrig, H., Ehrig, K., de Lara, J., Taentzer, G., Varró, D., & Varró, G. (2005).** *"Termination and confluence of graph transformation systems."* **Formal Methods in Software and Systems Modeling**, LNCS 3440, pp. 180–195. DOI: [10.1007/978-3-540-31847-7_11](https://doi.org/10.1007/978-3-540-31847-7_11)
- **Heckel, R., & Taentzer, G. (2020).** *Graph Transformation for Software Engineers: Formal Foundation and Industrial Applications.* Springer. DOI: [10.1007/978-3-030-43969-9](https://doi.org/10.1007/978-3-030-43969-9)

ModelScript Rationale & Modifications

In polyglot workflows, multiple rules may compete to transform the same AST elements. CPA statically detects all critical pairs (conflicting rule overlaps, target collisions, and circular translation loops) at compile time.

- **Modifications:** Tailored for heterogeneous polyglot metamodels; emits structured diagnostics with source ranges and rule conflict identifiers directly to the LSP and CLI.

3. Worst-Case Optimal Joins (WCOJ / Leapfrog Triejoin)

Academic Citations

- **Ngo, H. Q., Porat, E., Ré, C., & Rudra, A. (2012).** *"Worst-case optimal join algorithms."* ACM PODS '12, pp. 37–48. DOI: [10.1145/2213556.2213565](https://doi.org/10.1145/2213556.2213565)
- **Veldhuizen, T. L. (2014).** *"Leapfrog Triejoin: A simple, worst-case optimal join algorithm."* ICDT '14, pp. 96–106. DOI: [10.4230/LIPIcs.ICDT.2014.96](https://doi.org/10.4230/LIPIcs.ICDT.2014.96)
- **Atserias, A., Grohe, M., & Marx, D. (2008).** *"Size bounds and query plans for relational joins."* IEEE FOCS '08, pp. 739–748. (The AGM Bound)

ModelScript Rationale & Modifications

Traditional binary relational joins suffer from exponential intermediate result explosions on cyclic subgraphs (e.g. triangle topologies between ports, connectors, and component definitions). WCOJ guarantees runtime strictly bounded by the Atserias-Grohe-Marx (AGM) bound, achieving $O(N^{3/2})$ for triangle queries rather than $O(N^2)$.

- **Modifications:** Formulates Leapfrog Triejoin iteration orders using descending vertex degree heuristics; emits execution plans directly into the AOT WebAssembly compilation engine.

4. Zero-GC E-Graphs & Equality Saturation

Academic Citations

- **Nelson, G., & Oppen, D. C. (1980).** *"Fast decision procedures based on congruence closure."* Journal of the ACM, 27(2), pp. 356–364. DOI: [10.1145/322186.322198](https://doi.org/10.1145/322186.322198)
- **Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009).** *"Equality saturation: A new approach to optimization."* ACM POPL '09, pp. 264–276. DOI: [10.1145/1480881.1480915](https://doi.org/10.1145/1480881.1480915)
- **Willsey, M., et al. (2021).** *"egg: Fast and extensible equality saturation."* ACM POPL '21, pp. 1–29. DOI: [10.1145/3434304](https://doi.org/10.1145/3434304)

ModelScript Rationale & Modifications

Avoids destructive phase-ordering in symbolic equation simplification. An E-Graph maintains exponential equivalence classes of mathematically equal expressions simultaneously, extracting the globally minimal expression before numerical simulation.

- **Modifications:** WebAssembly linear memory allocation with zero GC; 32-bit Union-Find with two-pass path compression; fixed power-of-two open-addressing hash table (`HASH_MASK = 65535`) for $O(1)$

deduplication.

5. Graph-Structured Stack (GSS) GLR Parsing

Academic Citations

- **Tomita, M. (1985).** *Efficient Parsing for Natural Language: A Fast Generalized LR Algorithm.* Kluwer Academic Publishers. DOI: [10.1007/978-1-4613-2621-2](https://doi.org/10.1007/978-1-4613-2621-2)
- **Scott, E., & Johnstone, A. (2006).** "Right Numerate Generalized LR Parsers." *ACM TOPLAS*, 28(4), pp. 577–618. DOI: [10.1145/1176894.1176896](https://doi.org/10.1145/1176894.1176896)
- **McPeak, S., & Nacula, G. C. (2004).** "Elkhound: A fast, practical GLR parser generator." *Compiler Construction (CC 2004)*, LNCS 2985, pp. 73–88. DOI: [10.1007/978-3-540-24723-4_6](https://doi.org/10.1007/978-3-540-24723-4_6)

ModelScript Rationale & Modifications

Handles ambiguous and non-deterministic engineering grammars (such as expression vs. component declaration ambiguities in Modelica and SysML v2) by splitting and merging parse paths via the GSS without backtracking.

- **Modifications:** Active stack heads reside in contiguous unmanaged buffers (`t_activeHeads`) with power-of-two hash probing for $O(1)$ stack head merging in WebAssembly.
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs:**
 - Raw source code parsed by the `WASM GLR Parser` .
 - Polyglot synchronization rules authored in `@modelscript/dsl` .
- **Downstream Consumers:**
 - Rewritten expressions feed directly into `DAEBuilder` .
 - TGG synchronizations feed bidirectional updates between SysML v2 and Modelica.

CHAPTER 33

Linear Memory Structures & Indexing

Implementation:

- Roaring Bitmaps: `RoaringBitmap` in `packages/runtime/src/wasm/core/roaring.ts`
- Front-Coded IRI Dictionary: `FrontCodedDictionary` in `packages/runtime/src/wasm/core/trie.ts`
- Slotted Page B+ Tree: `PagedBTree` in `packages/runtime/src/wasm/storage/paged_btree.ts`

Academic Citations

Compressed Roaring Bitmaps

- **Chambi, S., Lemire, D., Kaser, O., & Godin, R. (2016).** *"Better bitmap performance with Roaring bitmaps."* **Software: Practice and Experience**, 46(5), pp. 709–719.
DOI: [10.1002/spe.2325](https://doi.org/10.1002/spe.2325)
- **Lemire, D., Ssi-Yan-Kai, G., & Kaser, O. (2018).** *"Consistently faster and smaller compressed bitmaps with Roaring."* **Software: Practice and Experience**, 48(9), pp. 1615–1638.
DOI: [10.1002/spe.2560](https://doi.org/10.1002/spe.2560)

Front-Coded String & IRI Compression

- **Witten, I. H., Moffat, A., & Bell, T. C. (1999).** *Managing Gigabytes: Compressing and Indexing Documents and Images* (2nd ed.). Morgan Kaufmann.
ISBN: [1-55860-570-3](https://www.morgankaufmann.com/9781558605703)
- **Martínez-Prieto, M. A., Fernández, J. D., & Cánovas, R. (2012).** *"Compression of RDF Dictionaries."* **ACM Transactions on the Web (TWEB)**, 6(4), pp. 1–35.
DOI: [10.1145/2382636.2382639](https://doi.org/10.1145/2382636.2382639)
- **Brisaboa, N. R., Cánovas, R., Francisco, C. C., Martínez-Prieto, M. A., & Navarro, G. (2011).** *"K2-trees for compact Web graph representation."* **Information Systems**, 39, pp. 152–163.
DOI: [10.1016/j.is.2012.08.003](https://doi.org/10.1016/j.is.2012.08.003)

Slotted Page B+ Tree & SPO Triple Storage

- **Bayer, R., & McCreight, E. (1972).** *"Organization and maintenance of large ordered indexes."* **Acta Informatica**, 1(3), pp. 173–189.
DOI: [10.1007/BF00288683](https://doi.org/10.1007/BF00288683)
- **Comer, D. (1979).** *"The ubiquitous B-tree."* **ACM Computing Surveys**, 11(2), pp. 121–137.
DOI: [10.1145/356770.356776](https://doi.org/10.1145/356770.356776)

- **Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., & Schwarz, P. (1992).** "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging." **ACM Transactions on Database Systems**, 17(1), pp. 94–162.
DOI: [10.1145/128765.128770](https://doi.org/10.1145/128765.128770)
- **Neumann, T., & Weikum, G. (2008).** "RDF-3X: A RISC-style engine for RDF." **Proceedings of the VLDB Endowment**, 1(1), pp. 647–659.
DOI: [10.14778/1453856.1453927](https://doi.org/10.14778/1453856.1453927)

ModelScript Architectural Rationale

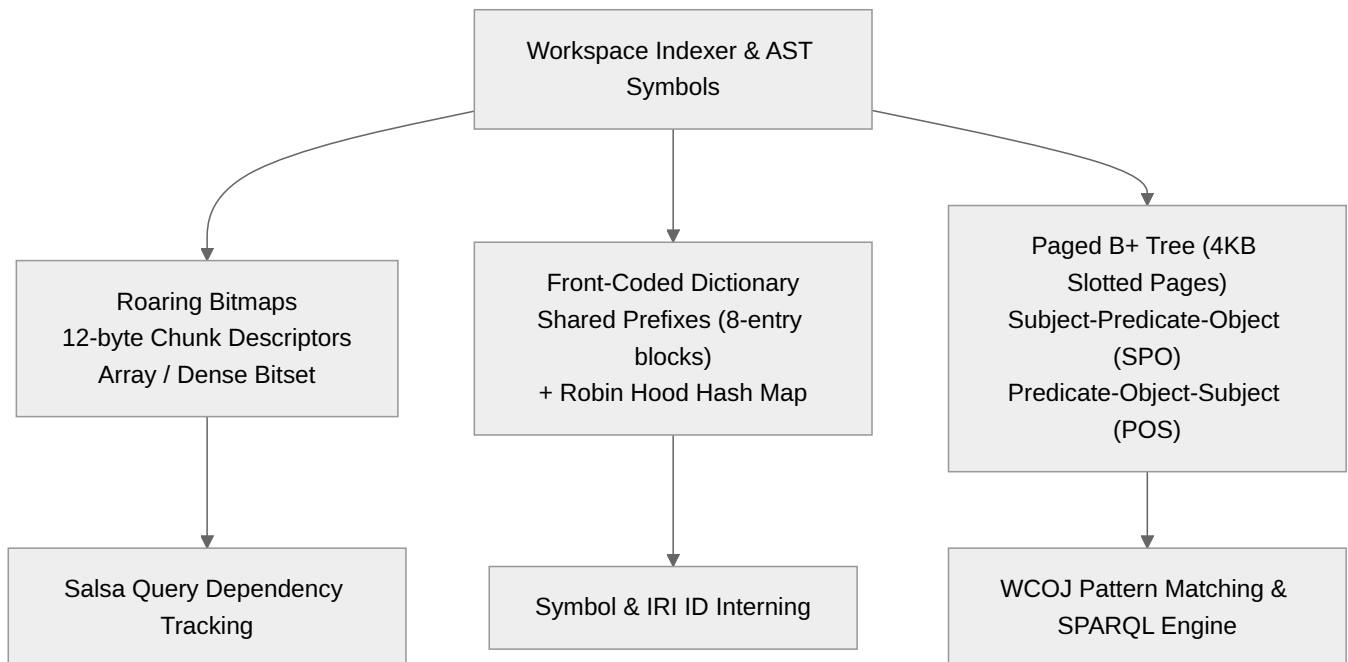
ModelScript compiles and indexes multi-million-element engineering workspaces, including the complete Modelica Standard Library (MSL), SysML v2 standard libraries (KerML), ISO 10303 STEP schemata, and OWL2 domain ontologies.

Representing these vast semantic graphs using standard JavaScript object heaps incurs:

- Extreme memory bloat (JavaScript objects typically consume 40–80 bytes of overhead per node).
- Heavy Garbage Collector (GC) pressure and unpredictable pause times during interactive IDE editing.
- Slow pointer chasing across cache lines.

To solve this, ModelScript implements **compact, zero-GC data structures** operating directly inside WebAssembly 64-bit linear memory.

Architecture of Core WASM Structures



1. Unmanaged Roaring Bitmaps

Partitions the 32-bit integer space into $2^{16} = 65,536$ chunks based on the upper 16 bits:

- **12-Byte Chunk Descriptor:**

```
Offset 0..3:   [key: u16] | [containerType: u16]
Offset 4..7:   [containerPtr: u32]
Offset 8..9:   [cardinality: u16]
Offset 10..11: [capacity: u16]
```

- **Adaptive Containers:**

- **Array Container:** Used when cardinality $< 4,096$. Elements are stored as sorted `u16` integers in linear memory.
- **Bitmap Container:** When cardinality reaches 4,096, the container automatically converts to a dense 8,192-byte bitset ($1,024 \times u64$ words), allowing bitwise `AND`, `OR`, `XOR` operations at hardware SIMD speeds.

2. Front-Coded String & IRI Dictionary

Engineering identifiers and semantic URIs often share massive common prefixes:

```
http://modelscrip.io/sysml2/core#PartDefinition
http://modelscrip.io/sysml2/core#ItemDefinition
http://modelscrip.io/sysml2/core#ActionDefinition
```

The front-coded dictionary partitions sorted strings into blocks of size $K = 8$:

- **Header String:** Stored uncompressed at the beginning of each block.
- **Suffix Entries:** Encoded as `[commonPrefixLen: u8, suffixLen: u8, suffixBytes]`.
- **Hybrid Lookups:** Combines front-coding (achieving 75–85% memory compression) with a zero-cost 64-bit Robin Hood hash map (`UnmanagedMap64`) for $O(1)$ string-to-ID interning.

3. Paged 4KB Slotted B+ Tree

Stores billions of semantic assertions and relational tuples:

- **4KB Slotted Page Layout (32-byte header + 254 entries):**

```
[0..3]   pageId: u32
[4..5]   pageType: u16 (1 = Leaf, 2 = Internal)
[6..7]   itemCount: u16
[8..11]  prevPage: u32 (doubly-linked leaf chain for range scans)
[12..15] nextPage: u32
[16..19] parentPage: u32
[20..23] flags: u32
[24..31] lsn: u64 (Log Sequence Number for crash consistency)
```

- **RDF-3X Triples:** Triples (S, P, O) are indexed in clustered B+ tree leaf chains. Full index permutations (`SP0` , `POS` , `OSP`) permit fast range scans for multi-way graph joins without intermediate hash joins.

Upstream & Downstream Integration

Data Structure	Upstream Dependencies	Downstream Consumers
RoaringBitmap	Symbol IDs from <code>SymbolIndexer</code>	Invalidation sets in <code>QueryEngine</code> , type sets
FrontCodedDictionary	AST string literals, Qualified names, URIs	Inter-file symbol resolution, memory-mapped LSP indexing
PagedBTree	AST semantic relationships, OWL2 axioms	Leapfrog Triejoin (WCOJ), graph query evaluators, SPARQL

CHAPTER 34

Scientific Computing, Linear Algebra & Surrogates

Implementation: `packages/runtime/src/wasm/solvers/` and `packages/simulate/src/surrogates/`

1. Davis SuiteSparse Left-Looking Cholesky (LL^T)

Academic Citations

- **Davis, T. A. (2006).** *Direct Methods for Sparse Linear Systems*. SIAM. DOI: [10.1137/1.9780898718881](https://doi.org/10.1137/1.9780898718881)
- **Chen, Y., Davis, T. A., Hager, W. W., & Rajamanickam, S. (2008).** "Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate." *ACM Transactions on Mathematical Software*, 35(3), pp. 22:1–22:14. DOI: [10.1145/1462166.1462176](https://doi.org/10.1145/1462166.1462176)

ModelScript Rationale & Modifications

Solves symmetric positive-definite sparse systems $Ax = b$ arising in finite element analysis, electrical mesh equations, and quadratic programming subproblems.

- **Modifications:** Left-looking column-by-column factorization computing non-zero patterns dynamically via elimination trees; operates inside unmanaged WebAssembly linear memory with zero heap fragmentation.

2. Gilbert-Peierls Left-Looking Sparse LU ($PAQ = LU$)

Academic Citations

- **Gilbert, J. R., & Peierls, T. (1988).** "An approach to sparse LU factorization based on graph coloring and topological sort." *International Conference on Supercomputing*.
- **Davis, T. A. (2006).** *Direct Methods for Sparse Linear Systems*. SIAM. Chapter 5.

ModelScript Rationale & Modifications

Factorizes unsymmetric, highly irregular sparse Jacobians generated by torn physical DAE systems.

- **Modifications:** Depth-first search (DFS) topological ordering to find the exact non-zero structure of each column in L and U in time proportional to arithmetic operations; threshold partial pivoting with diagonal preference to preserve physical sparsity.

3. Householder QR with Column Pivoting

Academic Citations

- **Golub, G. H., & Van Loan, C. F. (2013).** *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- **Businger, P., & Golub, G. H. (1965).** "Linear least squares solutions by Householder transformations." *Numerische Mathematik*, 7(3), pp. 269–276. DOI: [10.1007/BF01436084](https://doi.org/10.1007/BF01436084)

ModelScript Rationale & Modifications

Computes numerically stable orthogonal factorizations $AP = QR$ for rank-deficient least-squares calibration and parameter sensitivity studies.

4. Buchberger's Gröbner Basis Engine with Gebauer-Möller Criteria

Academic Citations

- **Buchberger, B. (1965).** *An Algorithm for Finding the Basis Elements of the Residue Class Ring of a Zero Dimensional Polynomial Ideal.*
PhD thesis, University of Innsbruck.
- **Gebauer, R., & Möller, H. M. (1988).** "On an installation of Buchberger's algorithm." *Journal of Symbolic Computation*, 6(2-3), pp. 275–286. DOI: [10.1016/S0747-7171\(88\)80048-8](https://doi.org/10.1016/S0747-7171(88)80048-8)
- **Cox, D., Little, J., & O'Shea, D. (2015).** *Ideals, Varieties, and Algorithms* (4th ed.). Springer. DOI: [10.1007/978-3-319-16721-3](https://doi.org/10.1007/978-3-319-16721-3)

ModelScript Rationale & Modifications

Solves small non-linear algebraic loops analytically to avoid numerical Newton iterations entirely. Computes the Gröbner basis under lexicographic monomial ordering (Lex) to triangularize polynomial systems into univariate root problems.

- **Modifications:** Implements the Gebauer-Möller criteria (B_{new} and M_{new} sets) to eliminate over 90% of redundant S-polynomial reduction checks; operates over exact rational coefficients Q .
-

5. Proper Orthogonal Decomposition (POD) / Snapshot SVD

Academic Citations

- **Sirovich, L. (1987).** "Turbulence and coherence in Catherine's wheel: Method of snapshots." *Quarterly of Applied Mathematics*, 45(3), pp. 561–571. DOI: [10.1090/qam/910462](https://doi.org/10.1090/qam/910462)
- **Berkooz, G., Holmes, P., & Lumley, J. L. (1993).** "The proper orthogonal decomposition in the analysis of turbulent flows." *Annual Review of Fluid Mechanics*, 25, pp. 539–575. DOI: [10.1146/annurev.fl.25.010193.002543](https://doi.org/10.1146/annurev.fl.25.010193.002543)

ModelScript Rationale & Modifications

Extracts reduced-order models (ROM) from massive 3D CFD or FEA simulation field snapshots ($N > 10^6$ nodes):

$$u(x, t) \approx \bar{u}(x) + \sum_{k=1}^r a_k(t) \phi_k(x)$$

- **Modifications**: Employs Sirovich's snapshot method to compute eigenvalues of the small $M \times M$ temporal correlation matrix ($M \ll N$ snapshots), retaining 99% cumulative kinetic energy in ≤ 16 spatial modes.

6. Buckingham π Dimensional Analysis & Similarity Engine

Academic Citations

- **Buckingham, E. (1914)**. "On physically similar systems; illustrations of the use of dimensional equations." *Physical Review*, 4(4), pp. 345–376. DOI: [10.1103/PhysRev.4.345](https://doi.org/10.1103/PhysRev.4.345)
- **Vaschy, A. (1892)**. "Sur les lois de similitude en physique." *Annales Télégraphiques*, 19, pp. 25–28.
- **Barenblatt, G. I. (1996)**. *Scaling, Self-Similarity, and Intermediate Asymptotics*. Cambridge Texts in Applied Mathematics 14. DOI: [10.1017/CBO9781107050242](https://doi.org/10.1017/CBO9781107050242)

ModelScript Rationale & Modifications

Reduces parameter dimensionality in surrogate modeling by converting raw physical quantities ($q_1, \dots, q_n$) into dimensionless similarity groups ($\Pi_1, \dots, \Pi_{n-r}$):

- **Modifications**: Computes the null-space of the 7-dimensional SI unit exponent matrix using exact rational Gaussian elimination with coprime integer exponents; canonical fingerprint matching recognizes standard dimensionless numbers (Reynolds Re , Nusselt Nu , Mach Ma , Froude Fr).
-

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
 - Linear matrix systems from `TornBlock` and `TR-BDF2`.
 - 3D CFD and FEA continuum field snapshots from `CFD` and `FEA`.
- **Downstream Consumers**:
 - Embedded into real-time Modelica plant loops via `msc surrogate`.
 - Accelerates linear solves across all numerical integrators and optimal control routines.

CHAPTER 35

Semidefinite Programming, BVP & Geometric CSG

Implementation:

- Primal-Dual Interior-Point SDP: `SdpSolver` in `packages/runtime/src/solvers/wasm_sdp_solver.ts`
- Multiple Shooting & Collocation BVP: `solveBvpShooting` in `packages/simulate/src/solvers/bvp-solver.ts`
- Constructive Solid Geometry & CFD Patches: `evaluateScad` in `languages/scad/src/evaluator.ts`

Academic Citations

Semidefinite Programming & Sum-of-Squares (SOS)

- **Vandenberghe, L., & Boyd, S. (1996).** *"Semidefinite Programming."*
SIAM Review, 38(1), pp. 49–95.
DOI: [10.1137/1038003](https://doi.org/10.1137/1038003)
- **Helmborg, C., Rendl, F., Vanderbei, R. J., & Wolkowicz, H. (1996).** *"An interior-point method for semidefinite programming."*
SIAM Journal on Optimization, 6(2), pp. 342–361.
DOI: [10.1137/0806018](https://doi.org/10.1137/0806018)
- **Alizadeh, F., Haeberly, J. P. A., & Overton, M. L. (1998).** *"Primal-dual interior-point methods for semidefinite programming: convergence rates, stability and numerical results."*
SIAM Journal on Optimization, 8(3), pp. 746–768.
DOI: [10.1137/S105262349528701X](https://doi.org/10.1137/S105262349528701X)
- **Parrilo, P. A. (2003).** *"Semidefinite programming relaxations for semialgebraic problems."*
Mathematical Programming, 96(2), pp. 293–320.
DOI: [10.1007/s10107-003-0387-5](https://doi.org/10.1007/s10107-003-0387-5) (SOS Programming)

Boundary Value Problems (BVP) & Multiple Shooting

- **Morrison, D. D., Riley, J. D., & Zancanaro, J. F. (1962).** *"Multiple shooting method for two-point boundary value problems."*
Communications of the ACM, 5(12), pp. 613–614.
DOI: [10.1145/368996.369028](https://doi.org/10.1145/368996.369028)
- **Stoer, J., & Bulirsch, R. (2002).** *Introduction to Numerical Analysis* (3rd ed.). Texts in Applied Mathematics 12, Springer. Section 7.3: Multiple Shooting Methods.
ISBN: [978-0-387-95452-3](https://doi.org/10.1007/978-0-387-95452-3)
- **Ascher, U. M., Mattheij, R. M., & Russell, R. D. (1995).** *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations.*
SIAM Classics in Applied Mathematics 13.
DOI: [10.1137/1.9781611971231](https://doi.org/10.1137/1.9781611971231)

- **Betts, J. T. (2010).** *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming* (2nd ed.). SIAM Advances in Design and Control 19. DOI: [10.1137/1.9780898718577](https://doi.org/10.1137/1.9780898718577)

Constructive Solid Geometry (CSG)

- **Requicha, A. A. (1980).** "Representations for rigid solids: Theory, methods, and systems." *ACM Computing Surveys*, 12(4), pp. 437–464. DOI: [10.1145/356827.356833](https://doi.org/10.1145/356827.356833)
- **Foley, J. D., Van Dam, A., Feiner, S. K., & Hughes, J. F. (1996).** *Computer Graphics: Principles and Practice* (2nd ed.). Addison-Wesley. Chapter 12: Constructive Solid Geometry. ISBN: [0-201-84840-6](https://www.addison-wesley.com/9780201848406)
- **Kienzle, M. (2009).** *OpenSCAD: The Programmers Solid 3D CAD Modeller*. <https://openscad.org>

ModelScript Architectural Rationale

1. **Semidefinite Programming (SDP) for Verified Safety:** Formal continuous verification of non-linear physical models requires proving that state trajectories $\mathbf{x}(t)$ never breach unsafe operating envelopes. Synthesizing polynomial Lyapunov functions $V(\mathbf{x}) > 0$ and barrier certificates $B(\mathbf{x}) \leq 0$ reduces to finding Sum-of-Squares (SOS) decompositions:

$$p(\mathbf{x}) = \mathbf{z}(\mathbf{x})^T Q \mathbf{z}(\mathbf{x}) \geq 0 \iff Q \succeq 0$$

By implementing a native primal-dual interior-point SDP solver in WebAssembly, ModelScript provides mathematically verified stability proofs directly in the browser and CLI without relying on external monolithic commercial solvers.

2. **Multiple Shooting BVP for Optimal Trajectories:** Boundary Value Problems (BVPs) govern optimal trajectory planning, model predictive control (MPC), and periodic limit cycle detection with split boundary conditions:

$$g(\mathbf{y}(t_0), \mathbf{y}(t_{\text{end}}), \mathbf{p}) = \mathbf{0}$$

Standard single shooting suffers from catastrophic numerical instability on stiff or chaotic dynamics. Multiple shooting subdivides the time horizon into M subintervals, guaranteeing numerical stability and fast Newton-Raphson convergence.

3. **Constructive Solid Geometry (CSG) for Multi-Physics 3D Packaging:** Lumped-parameter 1D physical models (Modelica) rely on 3D geometric properties (mass, inertia tensor, aerodynamic drag area, hydraulic volume). ModelScript evaluates OpenSCAD models into normalized CSG solid trees (`@modelscript/cad`), auto-tagging boundary patches (`inlet`, `outlet`, `wall`, `symmetry`) for downstream OpenCascade B-Rep evaluation, STEP export, and SU2/OpenFOAM CFD meshing.

Mathematical Formulation & Execution Architecture



Syntax error in text mermaid version 11.16.1

1. Primal-Dual Interior Point SDP Solver

Solves standard primal and dual semidefinite programs:

$$\text{Primal: } \min_{X \in \mathbb{S}^n} \text{tr}(CX) \quad \text{s.t.} \quad \text{tr}(A_i X) = b_i, \quad X \geq 0$$

$$\text{Dual: } \max_{y \in \mathbb{R}^m, S \in \mathbb{S}^n} b^T y \quad \text{s.t.} \quad \sum_{i=1}^m y_i A_i + S = C, \quad S \geq 0$$

- **Central Path Following**: Enforces the perturbed complementarity condition $XS = \mu I$ with barrier parameter $\mu = \frac{\text{tr}(XS)}{n}$. - **WASM Linear Algebra**: Predictor-corrector search directions $(\Delta X, \Delta y, \Delta S)$ are computed using fast in-memory Cholesky factorizations (`wasm_gaussian.ts`), with backtracking line search to ensure $X + \alpha \Delta X \succ 0$ and $S + \alpha \Delta S \succ 0$.

2. Multiple Shooting BVP Solver

Subdivides the horizon $[t_0, t_{\text{end}}]$ into M subintervals with mesh points $t_0 < t_1 < \dots < t_M = t_{\text{end}}$:

- **State Trajectory Variables**: Introduces free initial conditions $\mathbf{s}_m$ at each node t_m .
- **Subinterval Integration**: Integrates forward from t_m to t_{m+1} using adaptive `tsit5`:

$$\mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p})$$

- **Matching Residual System**: Solves the $(M \cdot n + k) \times (M \cdot n + k)$ simultaneous nonlinear system:

$$F_m(\mathbf{s}_m, \mathbf{s}_{m+1}) = \mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p}) - \mathbf{s}_{m+1} = \mathbf{0}, \quad m = 0, \dots, M-1$$

$$g(\mathbf{s}_0, \mathbf{s}_M, \mathbf{p}) = \mathbf{0}$$

- **Newton-Raphson Update**: Evaluates sensitivity Jacobians along trajectories and solves the block-banded linear system via WASM Gaussian LU elimination.

3. OpenSCAD CSG Engine & CFD Boundary Tagging

Transforms procedural CSG trees into boundary-tagged solid geometries:

- Evaluates transformations (`translate` , `rotate` , `scale` , `mirror`) and regularized Boolean operators (`union` , `difference` , `intersection`).
 - Decorates faces with CFD boundary tags (`tagPatch` , `BoundaryPatchType` : `inlet` , `outlet` , `wall` , `symmetry`), enabling seamless handoff to automated SU2 mesh generators.
-

Upstream & Downstream Integration

Component	Upstream Dependencies	Downstream Consumers
SdpSolver	Polynomial constraints from <code>TheoryCoordinator</code>	Formal Lyapunov barrier certificates, safety proofs
BvpSolver	ODE dynamics from <code>DaeBuilder</code> , split boundary specs	Model predictive control (MPC), trajectory optimization
ScadEvaluator	Declarative <code>.scad</code> ASTs, Modelica parameters	OpenCascade B-Rep kernels, STEP CAD export, CFD meshing

CHAPTER 36

Diagram Auto-Layout & Orthogonal Routing

Implementation:

- Topology-Shape-Metrics Engine: `TsmLayoutEngine` in `packages/diagram/src/tsm-layout.ts`
- Port Constraint & Alignment Solver: `PortIlpSolver` in `packages/diagram/src/port-ilp-solver.ts`
- Obstacle-Avoiding Channel Router: `PortRouter` in `packages/diagram/src/port-router.ts`

Academic Citations

Topology-Shape-Metrics (TSM / Kandinsky) Orthogonal Layout

- **Tamassia, R. (1987).** "On embedding a grid graph with minimum number of bends." *SIAM Journal on Computing*, 16(3), pp. 421–444.
DOI: [10.1137/0216030](https://doi.org/10.1137/0216030)
- **Föbmeier, U., & Kaufmann, M. (1996).** "Drawing high-degree graphs with minimum number of bends." *Graph Drawing (GD '95)*, Lecture Notes in Computer Science, 1027, pp. 211–225. Springer.
DOI: [10.1007/BFb0021804](https://doi.org/10.1007/BFb0021804)
- **Di Battista, G., Eades, P., Tamassia, R., & Tollis, I. G. (1998).** *Graph Drawing: Algorithms for the Visualization of Graphs*. Prentice Hall.
ISBN: [0-13-301615-3](https://www.amazon.com/dp/0133016153)

Port Constraint & Continuous Alignment ILP

- **Spönemann, M., von Hanxleden, R., & Fuhrmann, H. (2014).** "Port constraints in layout algorithms." *Graph Drawing (GD 2014)*, Lecture Notes in Computer Science, 8871, pp. 351–362. Springer.
DOI: [10.1007/978-3-662-45803-7_29](https://doi.org/10.1007/978-3-662-45803-7_29)
- **Kieffer, S., Dwyer, T., Marriott, K., & Wybrow, M. (2016).** "Incremental layout for dynamic schematic diagrams." *IEEE Transactions on Visualization and Computer Graphics*, 22(1), pp. 876–885.
DOI: [10.1109/TVCG.2015.2467364](https://doi.org/10.1109/TVCG.2015.2467364)
- **Eiglsperger, M., Föbmeier, U., & Kaufmann, M. (2001).** "Orthogonal graph drawing with constraints." *Journal of Graph Algorithms and Applications*, 5(2), pp. 1–24.
DOI: [10.7155/jgaa.00035](https://doi.org/10.7155/jgaa.00035)

Orthogonal Channel Routing & Obstacle Avoidance

- **Lee, C. Y. (1961).** "An algorithm for path connection and its applications." *IRE Transactions on Electronic Computers*, EC-10(3), pp. 346–365.
DOI: [10.1109/TEC.1961.5219222](https://doi.org/10.1109/TEC.1961.5219222)

- **Hart, P. E., Nilsson, N. J., & Raphael, B. (1968).** *"A formal basis for the heuristic determination of minimum cost paths."*
IEEE Transactions on Systems Science and Cybernetics, 4(2), pp. 100–107.
 DOI: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136) (*A* Search*)
 - **Deutsch, D. N. (1976).** *"A 'dogleg' channel router."*
Proceedings of the 13th Design Automation Conference (DAC '76), pp. 425–433.
 DOI: [10.1145/800263.810688](https://doi.org/10.1145/800263.810688)
-

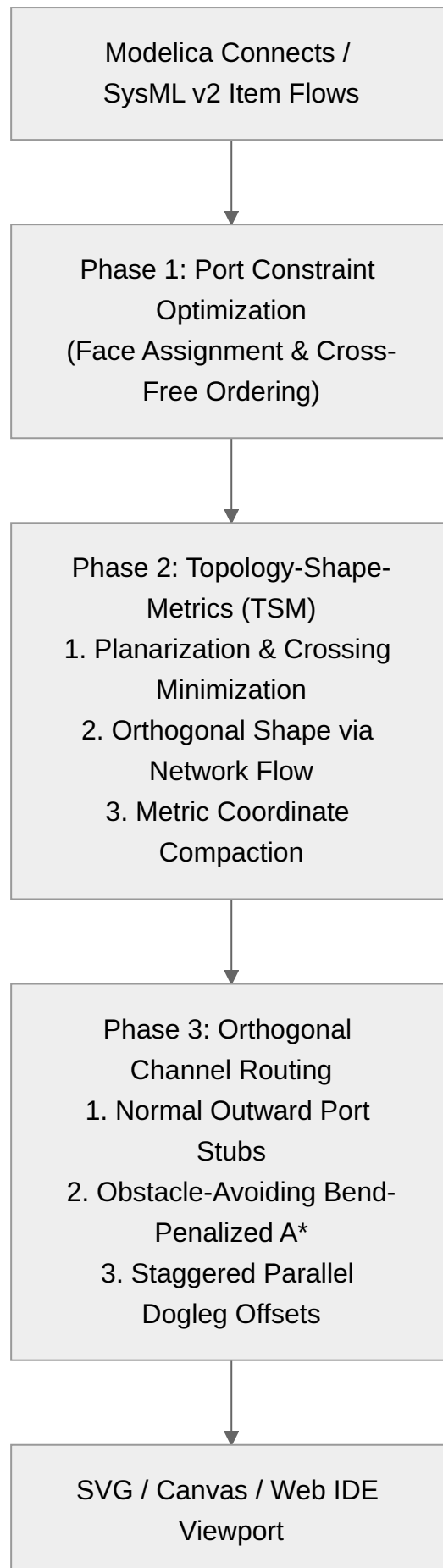
ModelScript Architectural Rationale

Physical engineering schematics (such as Modelica connection diagrams, SysML v2 Internal Block Diagrams, and hydraulic/piping schematics) require strict geometric conventions:

- **Orthogonal Edges:** Connections must run strictly along horizontal and vertical channels with clean 90° bends.
- **Port-Constraint Preservation:** High-degree engineering components have physical boundary pins (e.g. electrical resistors have pins p and n on opposite faces; hydraulic valves have ports A, B, P, T). Layout engines must place connection points on explicit component boundaries rather than abstract node centers.
- **Zero-Obstacle Penetration:** Wires must never traverse component bounding boxes.
- **Bus Disambiguation:** Multi-variable connections between blocks must not collapse onto identical collinear segments.

ModelScript implements the **Topology-Shape-Metrics (TSM)** pipeline augmented with a **Global Port ILP Solver** and **Dogleg Channel Router** natively in the browser and CLI.

Three-Phase Layout & Routing Architecture



1. Port Constraint & Continuous Alignment Solver

Resolves port assignment along component perimeters ($x, y \in \partial\text{Node}$):

- **Discrete Face Selection:** Assigns ports to Cardinal sides (`top` , `bottom` , `left` , `right`) minimizing Manhattan connection distance:

$$\min \sum_{e=(u_i, v_j) \in E} \|p(u_i) - p(v_j)\|_1$$

- **Causality Biasing:** Automatically places `input` ports on the left face and `output` ports on the right face for signal-flow blocks.
- **Collinear Port Snapping:** Enforces 0-bend straight lines whenever two connected ports are horizontally or vertically aligned within a configurable tolerance `straightTolerance` .

2. Topology-Shape-Metrics (TSM) Engine

Extends Tamassia's Kandinsky model for high-degree engineering vertices:

1. **Planarization:** Replaces unavoidable edge crossings with dummy vertices of degree 4.
2. **Orthogonalization (Minimum Bend Flow):** Formulates bend minimization as a minimum-cost network flow problem over the dual graph, assigning angles $\alpha \in \{90^\circ, 180^\circ, 270^\circ, 360^\circ\}$ to faces.
3. **Compaction:** Solves two independent 1D longest path problems on horizontal and vertical constraint DAGs to compute minimal integer grid coordinates.

3. Obstacle-Avoiding Dogleg Channel Router

Renders clean multi-segment connection polylines:

- **Perpendicular Port Stubs:** Edge paths extend outward along the port normal vector by a distance `stubLength` before turning.
- **A* Grid Search with Bend Penalty:** Avoids component AABB bounding boxes using fast `segmentIntersectsRect` intersection tests while penalizing unnecessary turns:

$$\text{Cost}(p \rightarrow q) = \text{dist}(p, q) + \lambda_{\text{bend}} \cdot [\text{isTurn}(p, q)]$$

- **Staggered Parallel Channels:** Multi-wire bus connections share a common corridor but are offset by `parallelIndex * channelSpacing` , preventing overlapping lines.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
PortIlpSolver	<code>SymbolIndex</code> connector ports, <code>Annotation</code> constraints	Port placement in <code>TsmLayoutEngine</code>
TsmLayoutEngine	Polyglot connect graphs (Modelica, SysML v2)	Node bounding box placement, routing channels
PortRouter	Component layout coordinates, Port locations	SVG polyline rendering, interactive 3D/2D IDE canvas

PART V

Command-Line Interface (msc)

CHAPTER 37

CLI Overview (`msc`)

The ModelScript unified command-line interface, `msc`, is published as `@modelscript/cli`.

It provides an all-in-one developer tool for parsing, linting, compiling, flattening, simulating, optimizing, and packaging engineering models.

Installation

Global Installation via npm

```
npm install -g @modelscript/cli
```

Direct Invocation via npx

```
npx @modelscript/cli --help
# Or when working inside the cloned ModelScript repository:
npx msc --help
```

Global Options

The following flags are available across all `msc` subcommands:

Flag	Description
<code>-v, --version</code>	Print the current CLI and compiler version.
<code>-h, --help</code>	Display help and usage instructions for the command.
<code>--verbose</code>	Enable detailed debug logs and execution traces.
<code>--quiet</code>	Suppress all informational output; only print errors or raw data payloads.
<code>--json</code>	Output machine-readable JSON payloads suitable for CI/CD scripting.
<code>--no-color</code>	Disable ANSI terminal color codes.

Interactive Playground Mode

Launch the interactive local CLI playground:

```
msc playground
```

This starts a terminal-based or local web UI for experimenting with models, viewing live equation trees, and inspecting flattened DAE structures in real time.

CHAPTER 38

CLI Command Reference

Comprehensive syntax, arguments, options, and usage examples for all primary `msc` subcommands.

`msc flatten`

Flattens a hierarchical Modelica model into a flat Differential Algebraic Equation (DAE) system.

```
msc flatten <model> [paths...] [options]
```

Options

- `--flattener <hybrid|wasm|ts|diff>` : Flattener backend (default: `hybrid`).
- `--no-fold` : Disable constant folding optimizations.
- `--no-alias` : Disable alias elimination.
- `--format <text|json>` : Output format (default: `text`).

Example

```
msc flatten Modelica.Electrical.Analog.Examples.ChuaCircuit path/to/MSL
```

`msc simulate`

Runs a time-domain numerical simulation of a Modelica model.

```
msc simulate <model> [paths...] [options]
```

Options

- `--start-time <t>` : Initial simulation time (default: `0.0`).
- `--stop-time <t>` : Final simulation time (default: `1.0`).
- `--step-size <h>` : Fixed or initial integrator step size.
- `--solver <cvode|ida|dopri5|rk4>` : Numerical integrator (default: `cvode`).
- `--tolerance <tol>` : Relative integration tolerance (default: `1e-6`).
- `--format <csv|json|parquet>` : Trajectory output format (default: `csv`).
- `--output <file>` : Destination file for simulation results.

Example

```
mssc simulate BouncingBall model.mo --stop-time 10 --solver ccode --output trajectory.csv
```

mssc optimize

Solves dynamic optimal control problems via direct collocation.

```
mssc optimize <model> [paths...] [options]
```

Options

- `--objective <expr>` : Objective function to minimize (e.g. `"u^2"`).
- `--controls <vars>` : Comma-separated list of control variables.
- `--control-bounds <var:min:max>` : Bound limits on control inputs.
- `--stop-time <t>` : Optimization horizon.

Example

```
mssc optimize InvertedPendulum model.mo \
  --objective "theta^2 + 0.1*u^2" \
  --controls "u" \
  --control-bounds "u:-10:10" \
  --stop-time 5
```

mssc lint

Runs static analysis and linter rules across Modelica and polyglot source files.

```
mssc lint [paths...] [options]
```

Options

- `--format <text|sarif|ctrf>` : Diagnostic output format.
- `--max-warnings <n>` : Maximum warning threshold before exiting with non-zero code.

Example

```
# Output SARIF format for GitHub Code Scanning
mssc lint models/ --format sarif > results.sarif
```

msc render

Renders Modelica annotations into interactive SVG diagrams.

```
msc render <model> [paths...] [options]
```

Options

- `--view <diagram|icon>` : Render schematic diagram or component icon (default: `diagram`).
- `--theme <light|dark|auto>` : Color theme (default: `auto`).

Example

```
msc render ChuaCircuit model.mo --view diagram > diagram.svg
```

msc fmu export

Exports a Modelica model as a standardized Functional Mock-up Unit (FMU).

```
msc fmu export <model> [paths...] [options]
```

Options

- `--version <2.0|3.0>` : FMI standard version (default: `3.0`).
- `--type <me|cs>` : Model Exchange (`me`) or Co-Simulation (`cs`).
- `--output <file.fmu>` : Destination archive path.

Example

```
msc fmu export DC_Motor motor.mo --version 3.0 --type cs --output DC_Motor.fmu
```

msc csg evaluate

Evaluates OpenSCAD scripts or STEP CAD solids.

```
msc csg evaluate <file.scad> [options]
```

Options

- `--output <file>` : Output format deduced from extension (`.step` , `.stl` , `.svg`).
- `--mass-properties` : Compute volume, mass center, and inertia tensor.

Example

```
msc csg evaluate bracket.scad --output bracket.step --mass-properties
```

msc surrogate

Trains or evaluates parametric reduced-order surrogate models.

```
msc surrogate <train|eval> [options]
```

Options

- `--model <fmu|mo>` : Input model or plant FMU.
- `--samples <n>` : Number of Latin Hypercube design samples (default: 500).
- `--method <pce|gp|ode>` : Polynomial Chaos, Gaussian Process, or Neural ODE.
- `--output <file.json>` : Serialized surrogate weights and hyper-parameters.

Example

```
msc surrogate train plant.fmu --samples 1000 --method gp --output surrogate.json
```

msc mc

Runs high-throughput Monte Carlo simulations under parameter uncertainty.

```
msc mc <model> [paths...] [options]
```

Options

- `--runs <n>` : Number of Monte Carlo trajectories (default: 1000).
- `--gpu` : Enable WebGPU acceleration for parallel execution.
- `--distribution <param:dist:args>` : Parameter uncertainty distributions.

Example

```
msc mc Circuit model.mo --runs 5000 --gpu --distribution "R:normal:100:5"
```

msc pr-diff

Computes semantic AST diffs between Git branches or revisions.

```
msc pr-diff <source-ref> <target-ref> [options]
```

Example

```
msc pr-diff main HEAD --models
```

PART VI

Reference & Diagnostics

CHAPTER 39

Compiler Diagnostics & Linter Rules

ModelScript enforces strict static semantic validation matching OpenModelica diagnostic conventions. Every diagnostic emitted by `@modelscrip/modelica` is assigned a unique numeric code (`M1xxx` through `M5xxx`) and kebab-case rule identifier.

Error Code Numbering Scheme

- `1xxx` : Syntax & Parser Errors
- `2xxx` : Name Resolution & Scoping
- `3xxx` : Type System & Compatibility
- `4xxx` : Structural Hierarchy & Declarations
- `5xxx` : Equations & Algorithms

1xxx: Syntax & Parse Diagnostics

M1001 : `parse-error`

- **Severity:** Error
- **Message:** `Parse error.`
- **Description:** The native WebAssembly GLR parser encountered tokens that do not conform to the Modelica grammar specification.
- **Remediation:** Check for missing semicolons, unbalanced parentheses, or misplaced keywords.

M1002 : `parse-missing`

- **Severity:** Error
- **Message:** `Parse error: '<token>' expected.`
- **Description:** A required grammatical delimiter (such as `;`, `)`, or `end`) was expected at the current token position.
- **Remediation:** Insert the missing delimiter.

M1003 : `empty-array-constructor`

- **Severity:** Error
- **Message:** `Empty array constructor '[]' or '{}'` is not valid in Modelica.
- **Description:** The Modelica language specification does not permit empty array literals without elements.
- **Remediation:** Provide at least one element or specify array dimensions via type attributes.

2xxx: Name Resolution Diagnostics

M2001 : duplicate-element

- **Severity:** Error
- **Message:** An element with name '<name>' is already declared in this scope.
- **Description:** A variable, parameter, or nested class with this identifier has already been defined in the current scope.
- **Remediation:** Rename or remove the redundant declaration.

M2002 : variable-not-found

- **Severity:** Error
- **Message:** Variable '<name>' not found in scope '<scope>'.
- **Description:** The identifier referenced in an expression or equation cannot be resolved in the local or inherited lexical scopes.
- **Remediation:** Verify the spelling of the variable or add an import/extends clause.

M2003 : class-not-found

- **Severity:** Error
- **Message:** Class '<className>' not found in scope '<scope>'.
- **Description:** The type specified in a component declaration or extends statement does not exist in the accessible package hierarchy.
- **Remediation:** Ensure the library containing <className> is included in the workspace paths.

M2004 : modifier-not-found

- **Severity:** Error
- **Message:** In modifier of '<componentName>', class or component '<modName>' not found in '<className>'.
- **Description:** A modifier clause attempts to override a parameter or component that is not declared in the target class.
- **Remediation:** Verify the parameter names available on <className>.

M2005 : identifier-mismatch

- **Severity:** Error
 - **Message:** The identifier at start and end are different.
 - **Description:** The class identifier declared at class <Name> does not match the closing identifier at end <Name>;.
 - **Remediation:** Update the ending identifier to match the class name.
-

3xxx: Type System Diagnostics

M3001 : type-mismatch-binding

- **Severity:** Error
- **Message:** Type mismatch in binding <name> = <expr>, expected subtype of <expected>, got type <actual>.
- **Description:** The evaluated type of a binding expression is incompatible with the declared component type (e.g. assigning a `String` to a `Real`).
- **Remediation:** Ensure the assigned expression evaluates to a compatible type or use explicit conversion functions.

M3002 : type-mismatch-modifier

- **Severity:** Error
- **Message:** Type mismatch: '<name>' expects type '<expected>' but got '<actual>'.
- **Description:** A modifier value assigned during component instantiation does not match the expected field type.
- **Remediation:** Correct the literal or expression supplied in the modifier parenthesis.

M3003 : not-plug-compatible

- **Severity:** Error
- **Message:** The connectors in `connect(<ref1>, <ref2>)` are not type compatible.
- **Description:** The two connector instances passed to `connect()` are not plug-compatible (different connector types or incompatible flow/stream variables).
- **Remediation:** Ensure both sides of `connect()` reference instances of the same connector definition.

M3004 : not-a-connector

- **Severity:** Error
- **Message:** In `connect(<ref1>, <ref2>):` '<which>' is not a connector.
- **Description:** An argument to `connect()` references a primitive scalar variable or standard class rather than a `connector`.
- **Remediation:** Connect equations can only link instances of classes declared with the `connector` restriction.

M3006 : function-arg-type-mismatch

- **Severity:** Error
- **Message:** Type mismatch for positional argument <pos> in <call>. The argument has type: <actual> expected type: <expected>.
- **Description:** An argument passed to a function call violates the signature expected by the function definition.
- **Remediation:** Verify the order and types of arguments in the function invocation.

4xxx: Structural & Hierarchy Diagnostics

M4001 : extends-cycle

- **Severity:** Error
- **Message:** extends <baseName> causes an instantiation loop.
- **Description:** A circular inheritance dependency was detected (e.g. *A* extends *B* extends *A*).
- **Remediation:** Break the inheritance cycle by refactoring shared elements into a common base class.

M4002 : duplicate-modification

- **Severity:** Error
- **Message:** Duplicate modification of element <name> on <kind> <compName>.
- **Description:** The same parameter or sub-element is modified multiple times in a single instantiation clause.
- **Remediation:** Remove the redundant modifier assignment.

M4004 : unbalanced-model

- **Severity:** Warning
 - **Message:** The <kind> '<name>' is not balanced: <nEquations> equation(s) and <nVariables> variable(s).
 - **Description:** A non-partial simulation model must have an equal number of scalar equations and non-parameter variables to be solvable.
 - **Remediation:** Add missing equations or declare surplus variables as `parameter` or `input`.
-

5xxx: Equations & Algorithms Diagnostics

M5001 : equation-type-mismatch

- **Severity:** Error
- **Message:** Type mismatch in equation <lhs>=<rhs> of type <lhsType>=<rhsType>.
- **Description:** The left-hand side and right-hand side expressions in an equality equation evaluate to incompatible types (e.g. equating a `Boolean` to a `Real`).
- **Remediation:** Ensure both expressions in the equation evaluate to compatible types.

CHAPTER 40

Standards & Export Formats

ModelScript bridges modern engineering workflows with extensive export and auditing formats, ensuring compatibility with standard enterprise tools, regulatory bodies, and CI/CD pipelines.

1. Functional Mock-up Interface (FMI 2.0 & 3.0)

ModelScript provides full-featured import and export for the **Functional Mock-up Interface (FMI)** standard via `@modelscript/exchange`:

```
# Export as FMI 3.0 Co-Simulation FMU
npx msc fmu export DC_Motor motor.mo --version 3.0 --type cs --output DC_Motor.fmu
```

Supported FMI Profiles

- **FMI 2.0 / 3.0 Model Exchange (ME)**: Standalone C or WebAssembly DAE evaluation functions; numerical integration is handled by the importing master environment.
- **FMI 2.0 / 3.0 Co-Simulation (CS)**: Bundles the SUNDIALS CVODE/IDA solver directly within the FMU container, advancing internal time steps autonomously.
- **FMI 3.0 Scheduled Execution (SE)**: Supports deterministic, clock-triggered discrete step execution for real-time controllers.

2. System Structure and Parameterization (SSP 1.0)

Packages complete multi-FMU simulation systems into standardized `.ssp` archives according to the Modelica Association standard:

- `SystemStructureDescription.xml`: Defines FMU component instances and signal connections.
- `SystemStructureParameterValues.xml`: Encapsulates calibration sets and parameter variations.
- `SystemStructureParameterMapping.xml`: Applies linear transformation scalings across connected signals.

3. Static Analysis Results Interchange Format (SARIF)

ModelScript linter and verification diagnostics can be exported in OASIS **SARIF v2.1.0** format:

```
npx msc lint models/ --format sarif > linter-report.sarif
```

- Direct ingestion into **GitHub Code Scanning** and Azure DevOps pull request checks.

- Generates inline code annotations with rule identifiers (`M1001` - `M5001`), source locations, and suggested fixes.

4. Common Test Report Format (CTRF)

For automated test suites and regression runners, ModelScript outputs **CTRF** reports (`ctrf-report.json`):

```
npx msc test --format ctrf --output ctrf-report.json
```

- Compatible with CTRF GitHub Actions, GitLab CI, and CircleCI dashboards.
- Records test duration, pass/fail counts, assertion diagnostics, and flakiness metrics.

5. Formal LaTeX Verification Proofs

When safety-critical systems require formal certification (e.g. ISO 26262, DO-178C), ModelScript's Nelson-Oppen Theory Coordinator can export rigorous mathematical proofs in LaTeX:

```
npx msc verify SafetyConstraint system.mo --format latex > proof.tex
```

- Exports step-by-step CDCL(T) deductions, Unsat cores, and interpolants.
- Formats continuous reachability zonotope equations ready for academic or regulatory publication.

6. Time-Series Trajectory Formats

Numerical simulation results (`msc simulate`) can be streamed to three primary trajectory formats:

Format	Extension	Use Case
CSV	<code>.csv</code>	Human-readable inspection, spreadsheet analysis, and test bench comparison.
JSON	<code>.json</code>	Web applications, browser charting (e.g. Morsel), and REST API payloads.
Parquet	<code>.parquet</code>	High-performance columnar storage for massive parameter sweeps and machine learning pipelines.